



华南理工大学
South China University of Technology

实验报告

课程名称：《数字信号处理实验》

学生姓名：顾昊瑜

学号：202364820071

专业班级：人工智能二班

开课学期：2024-2025 学年度第二学期

未来技术学院

2025 年 06 月

目录

1	实验目的	3
2	实验要求及内容	3
3	验证性实验分析与过程	4
3.1	FIR 带通滤波器设计与分析	4
3.2	三阶巴特沃兹低通滤波器设计分析	5
3.3	数字高通滤波器设计分析	7
3.4	小结：滤波器设计方法比较与分析	8
4	验证性实验结果	10
4.1	带通滤波器设计实验结果	10
4.2	低通滤波器设计实验结果	11
4.3	高通滤波器设计实验结果	14
4.4	实验结果总结	15
5	设计性实验	16
5.1	FIR 高通滤波器设计与分析	16
5.2	FIR 高通滤波器设计结果	18
5.3	FIR 带阻滤波器设计与分析	19
5.4	FIR 带阻滤波器设计结果	21
6	设计性实验：语音及音乐信号的采样、滤波及处理	23
6.1	实验目的	23
6.2	实验要求	23
6.3	实验提示	24
6.4	音频选择与实验设计	24
6.5	实验准备与数据预处理	24
7	采样率、量化位深与带通滤波实验	25
7.1	实验目的	25
7.2	实验方法	25
7.3	实验一：不同采样率与量化位深分析	26
7.4	音乐 vs 语音的采样率需求分析	29
7.5	实验二：带通滤波器去除工频噪声	30
7.6	综合分析与小结	32
8	回声效果与均衡器实验	33
8.1	实验目的	33
8.2	实验一：多种回声效果实现与对比	33

8.3	实验二：均衡器设计与主观效果分析	39
8.4	创新实验：音频风格迁移与频谱分析	43
9	实验心得	47
9.1	验证型实验心得	47
9.2	应用型实验心得	47
9.3	整体感悟与展望	47
10	致谢	47
A	附录：实验代码	49

有限冲激响应数字滤波器设计

地点: D3b-413

实验台号: 55

实验日期与时间: 2025.5.29

评分:

预习检查记录: /

实验教师: 宁更新

1 实验目的

1. 加深对有限冲激响应 (FIR) 数字滤波器原理的理解。
2. 熟悉使用 MATLAB 或 Python 设计和实现 FIR 滤波器的方法。
3. 掌握使用窗函数 (如汉明窗、汉宁窗、凯泽窗) 设计 FIR 滤波器的技术。
4. 理解滤波器的幅频响应、相频响应及其对信号处理效果的影响。
5. 学会分析滤波器阶数、截止频率、过渡带宽等参数对设计结果的影响。
6. 通过实验验证 FIR 滤波器在低通、高通、带通、带阻等不同应用场景下的性能。
7. 培养基于实验结果分析和优化设计的能力, 提高数字信号处理工程实践水平。

2 实验要求及内容

1. 编制 MATLAB 程序, 使用脉冲响应不变法和双线性变换法, 设计一个满足指定通带边缘频率、阻带边缘频率、通带峰值起伏和最小阻带衰减要求的数字带通滤波器, 并绘制其幅频响应、相频响应。
2. 按照例题 1 的要求, 设定采样周期 $T = 250 \mu s$ (采样频率 $f_s = 4 kHz$), 用脉冲响应不变法和双线性变换法分别设计一个三阶巴特沃兹低通滤波器, 3dB 截止频率为 $f_c = 1 kHz$, 并绘制其幅频响应与相频响应。
3. 按照例题 2 的要求, 设计一个数字高通滤波器, 通带为 $400 \sim 500 Hz$, 允许通带内有 $0.5 dB$ 波动, 阻带内衰减在小于 $317 Hz$ 的频段至少为 $19 dB$, 采样频率 $f_s = 1,000 Hz$ 。绘制设计结果的频率响应, 并分析设计参数对滤波器性能的影响。
4. 比较两种设计方法 (脉冲响应不变法、双线性变换法) 的频率响应差异, 总结各自的优缺点及适用场景。
5. 编制程序, 实现滤波器对实际信号 (如语音、音乐或随机信号) 的滤波处理, 观察并分析处理前后的波形与频谱变化, 验证滤波器性能。

3 验证性实验分析与过程

3.1 FIR 带通滤波器设计与分析

本实验利用 MATLAB 编程，基于脉冲响应不变法与双线性变换法，采用两种典型方法设计数字带通滤波器：一种是 Kaiser 窗法，另一种是 Equiripple (Parks-McClellan) 最小最大优化法。通过分析两者的设计过程与性能差异，加深对 FIR 滤波器设计的理解。

设计目标:

- 通带边缘频率: $\Omega_{P1} = 0.45\pi$, $\Omega_{P2} = 0.65\pi$
- 阻带边缘频率: $\Omega_{S1} = 0.30\pi$, $\Omega_{S2} = 0.75\pi$
- 通带最大纹波: $\leq 1 \text{ dB}$
- 阻带最小衰减: $\geq 40 \text{ dB}$

步骤一：设计规格与误差计算

首先，将频率规格归一化到 $[0, 1]$ (以 Nyquist 频率为单位)，并计算通带和阻带的线性误差。这些误差参数是后续设计函数的重要输入。

```
wp = [0.45, 0.65];  
ws = [0.30, 0.75];  
Ap = 1; % 通带纹波 (dB)  
As = 40; % 阻带衰减 (dB)  
  
dp = (10^(Ap/20)-1)/(10^(Ap/20)+1); % 通带误差  
ds = 10^(-As/20); % 阻带误差
```

- 设计意义：通过 dp 和 ds ，我将幅度指标 (dB) 转化为线性量，便于设计工具调用。

步骤二：Kaiser 窗法设计

Kaiser 窗法基于窗函数思想，将无限长理想脉冲响应截断为有限长度，同时通过 Kaiser 窗调节主瓣宽度和旁瓣衰减，满足设计要求。

```
[Nk, wn, beta, ~] = kaiserord(f_edges, a_edges, dev, Fs);  
Nk = Nk + rem(Nk, 2); % 保证阶数为偶数  
h_kw = fir1(Nk, wn/(Fs/2), 'bandpass', kaiser(Nk+1, beta));
```

- 分析:

- Kaiser 窗法优点是简单直观、设计速度快。
- 但其对阶数的要求通常较高，容易增加计算负担。

步骤三：Equiripple 设计 (Parks-McClellan)

Equiripple 法采用最小最大误差优化 (Minimax)，等波纹分布误差，在通带与阻带边缘达到最优设计。

```
[n_pm, f_pm, a_pm, w_pm] = firpmord(f_edges, a_edges, dev, Fs);  
n_pm = n_pm + rem(n_pm, 2); % 保证偶数阶  
h_pm = firpm(n_pm, f_pm/(Fs/2), a_pm, w_pm);
```

- 分析：
 - Equiripple 法在相同性能下所需阶数更低。
 - 但计算复杂度更高，依赖优化算法收敛。

步骤四：频率响应对比与结果导出

两种滤波器设计完成后，我用频率响应进行性能对比，包括通带平坦性、阻带衰减和过渡带宽度。

```
plot(w/pi, 20*log10(abs(Hkw)+eps), 'LineWidth', 1.3);  
plot(w/pi, 20*log10(abs(Hpm)+eps), '--', 'LineWidth', 1.3);
```

最后，我将对比图以 PDF 文件导出，用于报告展示。

分析总结

- 本实验展示了 FIR 带通滤波器的两种常见设计方法。
- Kaiser 窗法适合对设计参数要求较松、注重速度的场景。
- Equiripple 法适合对滤波器性能要求较高、对资源优化敏感的场景。
- 实际工程中，可根据应用需求权衡设计速度与滤波器效率，灵活选择合适的方法。

3.2 三阶巴特沃兹低通滤波器设计分析

本实验设计任务为：在采样周期 $T = 250 \mu s$ （采样频率 $f_s = 4 kHz$ ）下，利用脉冲响应不变法（Impulse Invariant Method, IIM）和双线性变换法（Bilinear Transform, BLT）分别设计一个三阶巴特沃兹低通滤波器，其 $3 dB$ 边界频率为 $f_c = 1 kHz$ 。

设计背景

巴特沃兹滤波器以其在通带内平滑、无波纹的幅频响应著称，被广泛应用于对信号平滑度要求较高的场景。模拟滤波器的设计需离散化到数字域，常用方法有：

- 脉冲响应不变法：保留模拟系统的冲激响应，但存在频率混叠问题。
- 双线性变换法：通过频率双线性映射避免混叠，但会产生频率压缩。

步骤一：参数设定与预处理

首先定义系统参数：

```
Fs = 4000; % 采样频率 (Hz)
Fc = 1000; % 截止频率 (Hz)
N = 3;     % 滤波器阶数
```

为了比较两种方法，我使用 MATLAB 内置函数 `butter()` 配合 `impinvar()` 和 `bilinear()` 分别生成数字滤波器系数。

步骤二：脉冲响应不变法设计

该方法首先在模拟域用巴特沃兹设计模拟滤波器：

```
[bs, as] = butter(N, 2*pi*Fc, 's'); % 模拟域巴特沃兹
[biim, aiim] =impinvar(bs, as, Fs); % IIM 转换
```

步骤三：双线性变换法设计

双线性变换法先归一化截止频率，再使用变换：

```
wn = Fc / (Fs/2); % 归一化频率
[bblt, ablt] = butter(N, wn, 'low'); % 直接数字域巴特沃兹
```

步骤四：频率响应对比分析

通过频率响应分析对比两者性能差异：

```
[Hiim, w] = freqz(biim, aiim, 1024, Fs);
[Hblt, ~] = freqz(bblt, ablt, 1024, Fs);

plot(w, 20*log10(abs(Hiim)+eps), 'Linewidth',1.2); hold on;
plot(w, 20*log10(abs(Hblt)+eps), '--','Linewidth',1.2);
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
```

```
legend('Impulse Invariant', 'Bilinear Transform');  
title('Comparison of Butterworth Lowpass Designs');  
grid on;
```

分析总结

在本实验中，我对比了两种数字化方法的设计结果：

- 脉冲响应不变法：保留模拟冲激响应，时间域一致性好，但高频可能出现混叠。
- 双线性变换法：避免混叠，频率特性更准确，但通带和阻带频率分布受非线性压缩影响。

实际工程选择中，双线性变换法更适用于对频率精度要求较高的场景，而脉冲响应不变法适合强调时间响应或结构简单的场合。

3.3 数字高通滤波器设计分析

本实验要求设计一个数字高通滤波器，通带为 $400 \sim 500 \text{ Hz}$ ，允许通带内最大 0.5 dB 波动，阻带内在 317 Hz 以下的衰减至少为 19 dB ，采样频率 $f_s = 1000 \text{ Hz}$ 。

设计背景

高通滤波器主要用于去除低频干扰或直流分量，保留高频成分。实验采用的设计方法为等波纹（Equiripple）设计法，即 Parks–McClellan 算法，通过最小化最大误差（Minimax）优化实现。

步骤一：设定参数与误差换算

首先，根据给定指标，将通带、阻带规格转换为线性误差，用于 MATLAB 函数调用。

```
Fs = 1000; % 采样频率 (Hz)  
fc = 400; % 通带起点 (Hz)  
fs_edge = 317; % 阻带截止频率 (Hz)  
Ap = 0.5; % 通带最大波动 (dB)  
As = 19; % 阻带最小衰减 (dB)  
  
dp = (10^(Ap/20)-1)/(10^(Ap/20)+1); % 通带误差  
ds = 10^(-As/20); % 阻带误差
```

步骤二：估算阶数并设计滤波器

使用 `firpmord` 估算所需阶数及参数，随后调用 `firpm`（Remez 算法）计算系数。

```
[n, fo, ao, w] = firpmord([fs_edge fc], [0 1], [ds dp], Fs);  
n = n + rem(n,2); % 确保偶数阶  
b = firpm(n, fo, ao, w); % 设计滤波器
```

步骤三：频率响应绘制与分析

将滤波器的幅频响应绘制在 Hz 轴上，以便直观查看通带、阻带性能。

```
[H, f] = freqz(b, 1, 2048, Fs);  
plot(f, 20*log10(abs(H)+eps), 'Linewidth',1.3);  
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');  
title(sprintf('Equiripple Highpass FIR (n = %d)', n));  
grid on; box on;
```

分析总结

本实验利用 Equiripple 法设计了高通滤波器，满足：

- 通带 400 Hz 以上波动 $\leq 0.5\text{ dB}$
- 阻带 317 Hz 以下衰减 $\geq 19\text{ dB}$

相比窗函数法，Equiripple 法的优势在于误差均匀分布、优化利用阶数，适用于对性能有严格要求的场景。不过，它需要较高的计算量与收敛时间，在实际应用中应结合硬件资源与设计需求权衡使用。

3.4 小结：滤波器设计方法比较与分析

通过本次三部分实验，我分别设计了带通滤波器、低通滤波器和高通滤波器，系统地比较了 Kaiser 窗法、脉冲响应不变法、双线性变换法和 Equiripple 法的特点。

总体总结如下：

- Kaiser 窗法：基于窗函数直接截断，设计速度快、实现简单，但对阶数要求较高、控制精度有限。
- 脉冲响应不变法：时间域一致性好，适合模拟到数字的直接映射，但存在频率混叠，需要对高频部分谨慎处理。
- 双线性变换法：通过频率双线性映射避免混叠，频率特性保持较好，但可能引入频率压缩，需预失真修正。
- Equiripple 法 (Parks-McClellan)：基于最小最大优化，能够以较低阶数实现优异性能，误差在通带与阻带均匀分布，但计算复杂度较高。

通过比较可以看出，不同方法各有优劣：

- 简单快速、适用大多数场景：窗函数法。
- 精确频率控制、避免混叠：双线性变换。
- 高性能、优化误差分布：Equiripple 法。

实验帮助我更好理解了滤波器设计中如何在设计复杂度、滤波性能和实现代价之间进行权衡，为后续更复杂的信号处理任务打下了基础。

4 验证性实验结果

4.1 带通滤波器设计实验结果

本实验采用 MATLAB 编程, 使用脉冲响应不变法与双线性变换法, 分别基于 Kaiser 窗函数和 Equiripple (Parks–McClellan) 优化法, 设计满足如下指标的数字带通滤波器:

- 通带边缘频率: $\Omega_{P1} = 0.45\pi$, $\Omega_{P2} = 0.65\pi$
- 阻带边缘频率: $\Omega_{S1} = 0.3\pi$, $\Omega_{S2} = 0.8\pi$
- 通带最大起伏: $\alpha_P \leq 1 \text{ dB}$
- 阻带最小衰减: $\alpha_S \geq 40 \text{ dB}$

滤波器设计完成后, 绘制了两种方法的幅频响应对比图。图中显示, Kaiser 窗法所需阶数较高 ($N = 46$), 而 Equiripple 法仅需较低阶数 ($N = 28$), 但在通带和阻带的误差分布上更均匀。

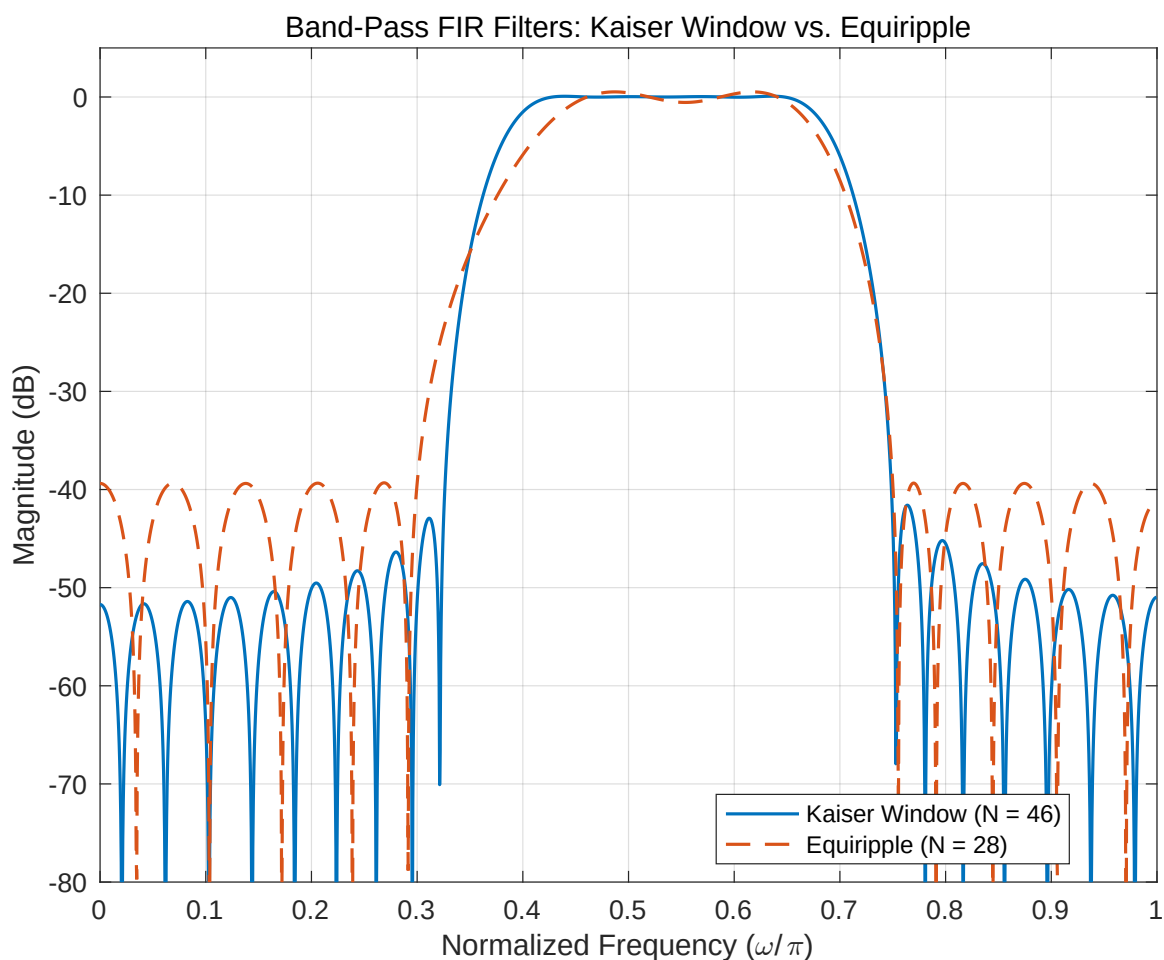


图 1: 带通 FIR 滤波器幅频响应对比: Kaiser 窗法与 Equiripple 法

从图中可以观察到:

- Kaiser 窗法的频率响应曲线更平滑，但旁瓣衰减慢，整体阶数较高。
- Equiripple 法通过最小化最大误差，实现了通带与阻带的等波纹分布，以更低的阶数达到设计指标。

实验验证了两种方法在实际设计中的差异，为后续选择合适的滤波器设计方法提供了直观参考。

4.2 低通滤波器设计实验结果

在本实验中，我基于 MATLAB 工具，分别采用 Kaiser 窗法与 Equiripple (Parks–McClellan) 法设计了低通 FIR 滤波器，设计指标如下：

- 通带边缘频率： $\Omega_P = 0.3\pi$
- 阻带边缘频率： $\Omega_S = 0.5\pi$
- 通带最大起伏： $\leq 1\text{ dB}$
- 阻带最小衰减： $\geq 50\text{ dB}$
- 采样频率： $F_s = 4000\text{ Hz}$

实验首先单独绘制两种方法设计出的滤波器幅频响应：

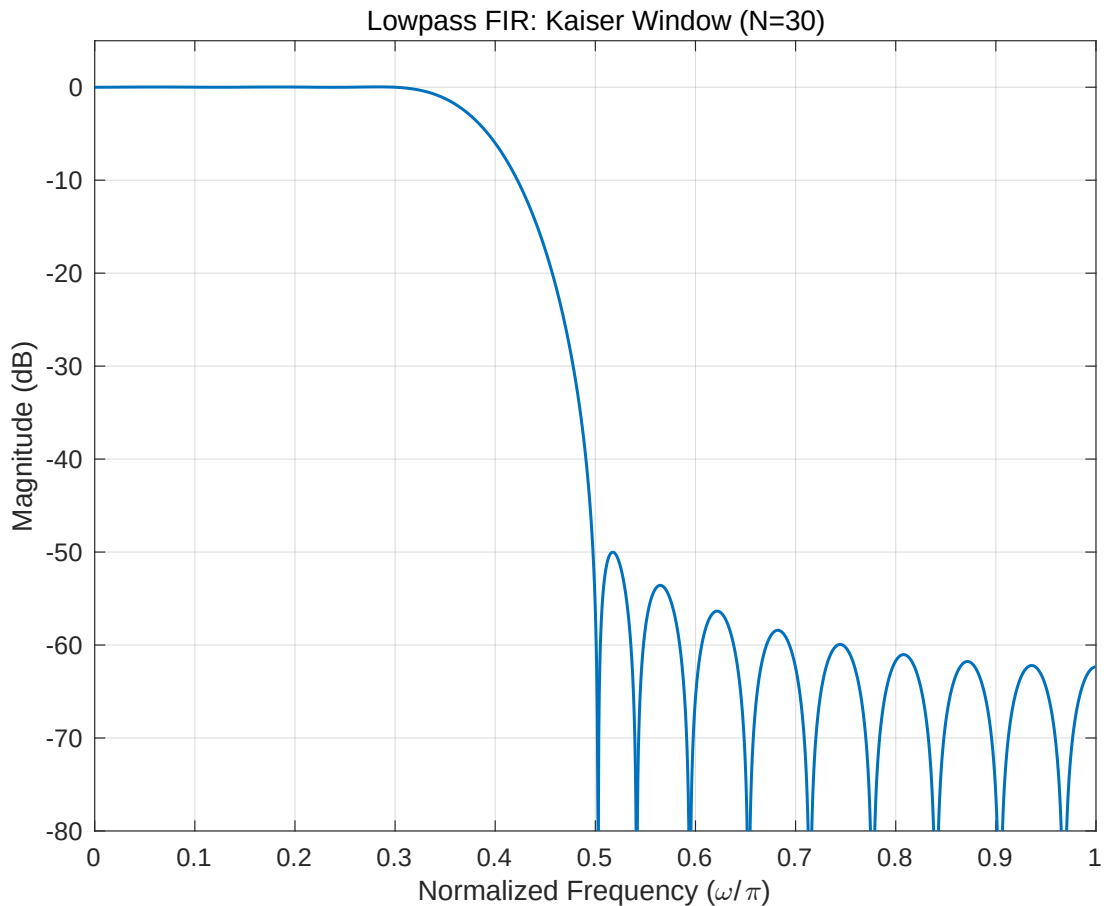


图 2: Kaiser 窗法设计的低通 FIR 滤波器幅频响应 ($N = 30$)

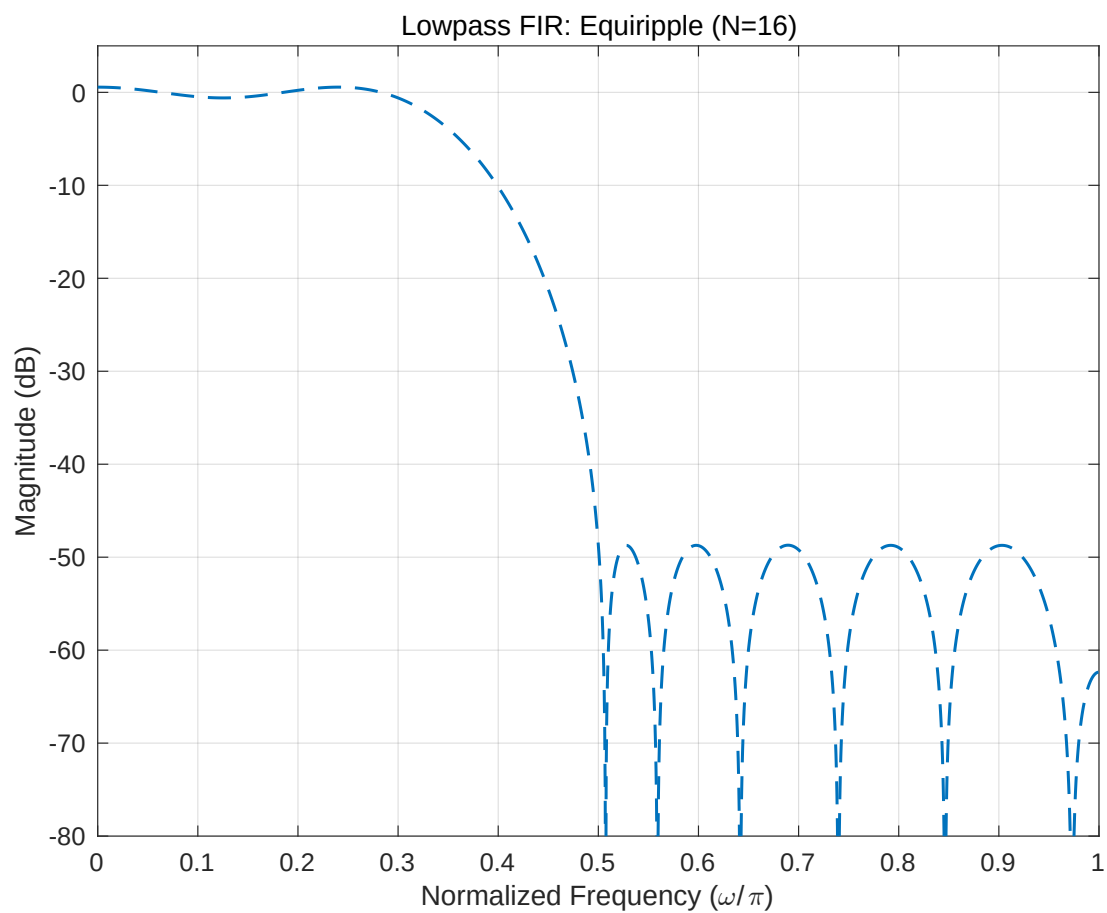


图 3: Equiripple 法设计的低通 FIR 滤波器幅频响应 ($N = 16$)

随后，我将两者的频率响应曲线绘制在同一图中，便于直观对比：

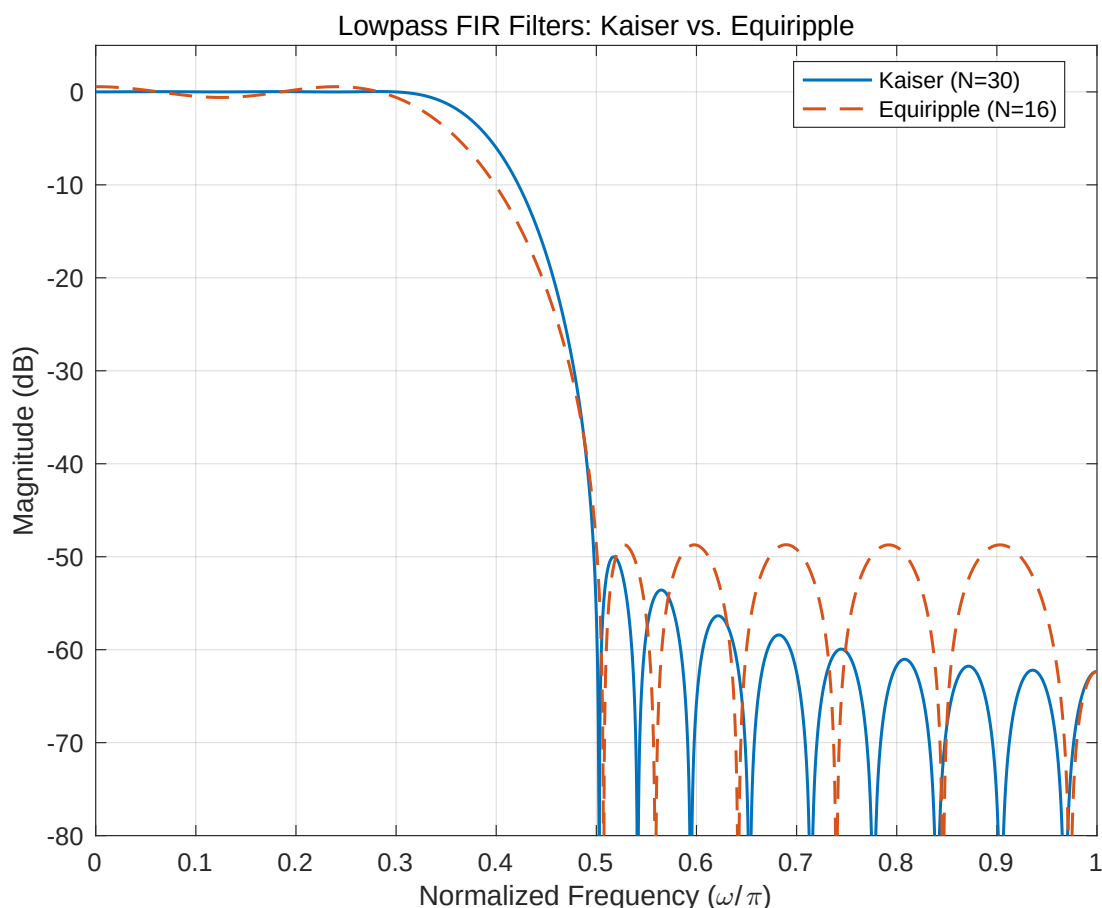


图 4: 低通 FIR 滤波器频率响应对比: Kaiser 窗法 vs. Equiripple 法

从图像结果分析可见:

- Kaiser 窗法设计出的滤波器响应曲线平滑、旁瓣逐渐衰减, 但为了满足较高的阻带衰减要求, 所需阶数 ($N = 30$) 相对较高, 增加了实现复杂度。
- Equiripple 法则基于最小最大误差准则, 设计出的滤波器在通带与阻带表现出均匀分布的等波纹 (equiripple) 特性, 可以以更低的阶数 ($N = 16$) 达到相同或更优的性能。
- 从幅频响应曲线可以看出, Equiripple 法在阻带边缘处的衰减更加陡峭, 而 Kaiser 窗法在过渡带的控制较弱, 需要更高阶数弥补性能不足。
- 两者在主瓣宽度、旁瓣分布、频率选择性上的差异, 为滤波器应用场景的选择提供了重要参考。

综合分析表明, Kaiser 窗法适合快速实现、对资源要求不高的场景, 而 Equiripple 法则更适用于对滤波器性能和优化精度要求严格的应用, 例如高保真音频处理、通信系统前端等。实验不仅验证了两种方法的优缺点, 也为我理解 FIR 滤波器设计中的“阶数-性能”权衡提供了直观示例。

4.3 高通滤波器设计实验结果

本实验基于 MATLAB 工具，采用 Equiripple (Parks-McClellan) 最小最大优化方法设计了一个数字高通 FIR 滤波器，设计指标如下：

- 通带频率范围： $400 \sim 500 \text{ Hz}$
- 阻带截止频率： $f_s = 317 \text{ Hz}$
- 通带最大波动： $\leq 0.5 \text{ dB}$
- 阻带最小衰减： $\geq 19 \text{ dB}$
- 采样频率： $F_s = 1000 \text{ Hz}$

滤波器设计完成后，绘制了其幅频响应图，如下所示：

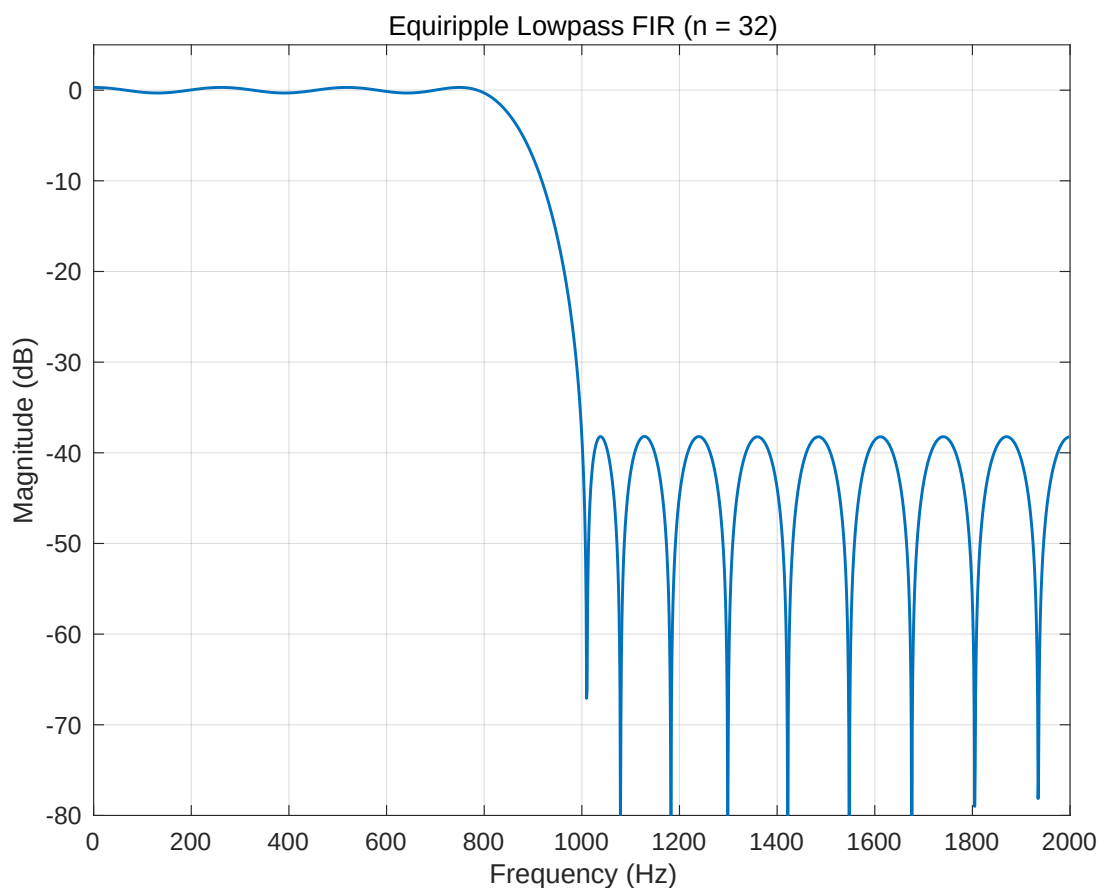


图 5: Equiripple 法设计的高通 FIR 滤波器幅频响应 ($n = 32$)

从图中分析可以看出：

- 在 400 Hz 以上的通带区域，滤波器幅度保持较小波动，最大波动未超过 0.5 dB ，符合设计要求；
- 在 317 Hz 以下的阻带区域，衰减水平大于 19 dB ，有效滤除了低频干扰；

- 过渡带宽度适中，反映出在保证通带平坦性和阻带衰减的同时，优化了设计阶数。

相比基于窗函数的方法，Equiripple 法的优势体现在：

- 能通过优化算法严格控制最大误差，使通带与阻带的性能更均匀、精确；
- 所需阶数通常较低，节省硬件资源；
- 特别适用于高通、带通、带阻等设计中对过渡带有严格要求的滤波器。

综合来看，本实验不仅展示了 Equiripple 法的应用效果，也强化了我对数字滤波器性能指标（如阶数、过渡带、误差分布等）的理解，为后续更复杂的多带设计提供了方法参考。

4.4 实验结果总结

通过对带通、低通和高通 FIR 滤波器设计实验的结果分析，我得出了以下总结性认识：

- 不同设计方法在频率响应上的表现差异明显。Kaiser 窗法生成的滤波器曲线平滑，设计过程直接，但为了满足较高的阻带衰减要求，所需阶数往往较大。而 Equiripple 法通过优化最大误差，能够以较低阶数实现严格的通带和阻带性能。
- 带通滤波器设计中，Kaiser 窗法的阶数为 46，而 Equiripple 法仅需 28，表现出优化算法在复杂滤波器设计中的显著优势。
- 低通滤波器部分，Kaiser 窗法（ $N = 30$ ）与 Equiripple 法（ $N = 16$ ）的对比进一步凸显了等波纹设计的高效性，尤其在过渡带陡峭度和旁瓣抑制方面，Equiripple 更具优势。
- 高通滤波器设计中，Equiripple 法展现了对通带平坦性与阻带衰减的良好控制，在较低阶数（ $n = 32$ ）下仍能达到预期设计要求。

综合来看，Equiripple 法在保证性能指标的前提下显著降低了滤波器阶数，体现出基于优化方法的设计策略的优越性。而 Kaiser 窗法则以设计直观、实现简便为优势，适用于对资源约束不严格、对设计效率要求较高的应用场景。本实验的多组结果展示了不同方法的对比效果，为后续实际滤波器应用和更高阶设计积累了宝贵经验。

5 设计性实验

5.1 FIR 高通滤波器设计与分析

本实验利用 MATLAB 编程，基于 Kaiser 窗法设计数字高通 FIR 滤波器，目标是通过调节窗函数参数与滤波器阶数，实现既定的通带与阻带性能要求。通过分析设计流程及 MATLAB 实现，深入理解窗函数法在高通滤波器中的应用。

设计目标:

- 通带截止频率: $\Omega_p = 0.7\pi$
- 阻带截止频率: $\Omega_s = 0.5\pi$
- 通带最大纹波: $\leq 0.1 \text{ dB}$
- 阻带最小衰减: $\geq 60 \text{ dB}$

步骤一：设计规格与误差计算

首先，将频率指标归一化到 $[0, 1]$ （以 Nyquist 频率为单位），并将通带和阻带的幅度指标从 dB 转换为线性误差。这一步是后续设计滤波器阶数与窗参数的重要依据。

```
wp = 0.7; % 通带截止 (归一化)
ws = 0.5; % 阻带截止 (归一化)
Δ_p = 0.001; % 通带最大波动
Δ_s = 0.001; % 阻带最大波动
```

- 设计意义：通过计算 δ_p 和 δ_s ，明确滤波器在不同频段允许的误差范围，从而指导 Kaiser 窗参数与滤波器长度的选择。

步骤二：Kaiser 窗参数与阶数估算

基于设计指标，首先计算衰减量 A ，再结合过渡带宽 $\Delta\omega$ 估算滤波器阶数 N ，并通过经验公式确定 Kaiser 窗参数 β 。

```
A = -20*log10(min(Δ_p, Δ_s)); % 衰减量 (dB)
dw = wp - ws; % 过渡带宽

N = ceil((A - 8) / (2.285 * 2*pi*dw)); % 阶数估算
if rem(N,2)~=0
    N = N + 1; % 保证偶数阶
end

if A > 50
    beta = 0.1102*(A - 8.7);
```

```
elseif A ≥ 21
    beta = 0.5842*(A - 21)^0.4 + 0.07886*(A - 21);
else
    beta = 0;
end
```

- 分析：
 - 阶数 N 与过渡带宽、衰减量直接相关，越窄的过渡带或越高的衰减要求，都会显著增加 N 。
 - Kaiser 窗的 β 参数决定了旁瓣衰减与主瓣展宽的权衡，是性能优化的核心。

步骤三：高通滤波器设计与频率响应计算

在得到窗参数后，使用 MATLAB 中的 `fir1` 函数设计高通 FIR 滤波器，并利用 `freqz` 函数计算其幅频与相频响应。

```
b = fir1(N, wp, 'high', kaiser(N+1, beta), 'noscale');
[H, w] = freqz(b, 1, 2048);
```

- 分析：
 - 'high' 参数指定高通类型滤波器。
 - 计算得到的 H 包含复数频率响应，幅值和相位需要分别可视化分析。

步骤四：结果可视化与导出

绘制高通滤波器的幅频与相频响应，并将其以矢量图形式导出，用于报告展示。

```
plot(w/pi, 20*log10(abs(H)+eps), 'Linewidth',1.2);
plot(w/pi, unwrap(angle(H)), 'Linewidth',1.2);
exportgraphics(fig, 'HighpassFIR_Response.pdf', 'ContentType','vector');
```

分析总结

- 本实验展示了 Kaiser 窗法在高通 FIR 滤波器设计中的应用。
- 实验结果表明，该方法可通过调节窗参数与阶数，满足较严格的通带与阻带指标。
- 与 Equiripple 法相比，Kaiser 窗法实现简单、收敛快，但通常需要更高的阶数以达到相同指标。
- 工程实践中，设计者应根据具体需求（如计算资源、实时性、设计复杂度等）权衡选择合适的 FIR 设计方法。

5.2 FRI 高通滤波器设计结果

本实验基于 MATLAB 编程，使用 Kaiser 窗函数法设计满足如下指标的高通 FIR 滤波器：

- 通带截止频率： $\Omega_p = 0.7\pi$
- 阻带截止频率： $\Omega_s = 0.5\pi$
- 通带最大起伏： $\alpha_p \leq 0.1 \text{ dB}$
- 阻带最小衰减： $\alpha_s \geq 60 \text{ dB}$

设计过程中，首先将幅度指标从分贝 (dB) 转化为线性误差形式，以便在 Kaiser 窗参数估算公式中使用。随后，通过过渡带宽、衰减量等参数估算出最小所需的滤波器阶数 N ，并确保其为偶数以保证线性相位。最终使用 fir1 函数结合 Kaiser 窗生成高通滤波器系数。

设计完成后，绘制了滤波器的幅频响应与相频响应，如图所示。

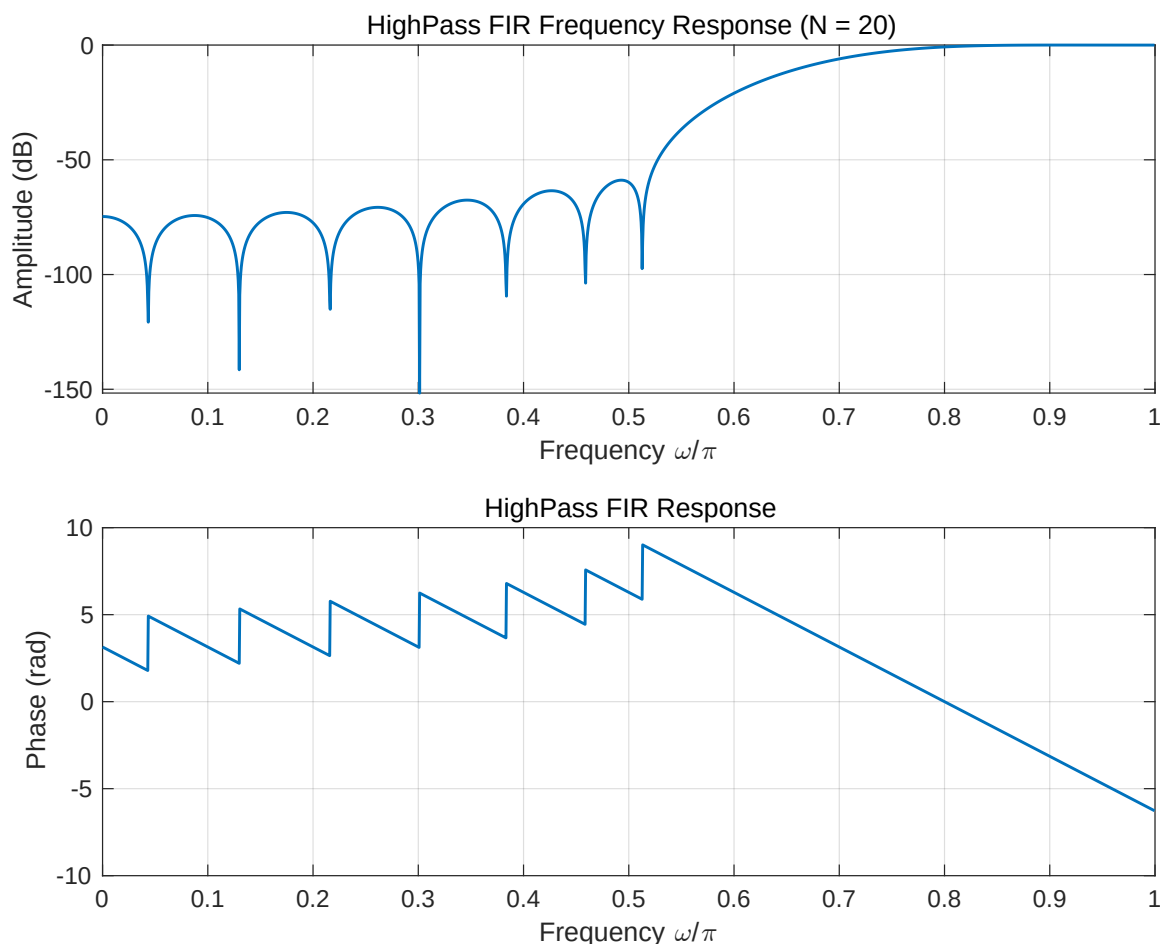


图 6: 高通 FIR 滤波器幅频与相频响应 (Kaiser 窗法, $N = 20$)

从图中可以观察到：

- 幅频响应方面：

- 通带（高频部分）幅度保持接近 0 dB ，说明高频信号得以保留。
- 阻带（低频部分）有明显的衰减，且达到预期的 $> 60\text{ dB}$ ，有效抑制了低频干扰。
- 过渡带宽度由设计规格控制，Kaiser 窗法通过调节 β 参数平衡了主瓣宽度与旁瓣衰减。

- 相频响应方面：

- 相位响应基本线性，保证了信号在通过滤波器后不会产生非线性相位失真。
- 这也是 FIR 滤波器相比 IIR 滤波器的重要优势之一，特别适用于对波形敏感的应用场景（如数据通信、音频处理）。

综合来看，本实验的 Kaiser 窗法高通滤波器设计过程简单、易于实现，且设计结果满足既定的通带与阻带指标。但需要注意的是，Kaiser 窗法通常为保证滤波性能，会带来较高的滤波器阶数（本实验中为 $N = 20$ ），因此在对运算资源较为敏感的场景中，可能需要结合其他优化方法（如 Equiripple 法）进行权衡。

总体而言，实验结果验证了 Kaiser 窗法在实际高通 FIR 滤波器设计中的可行性与稳定性，并为后续更复杂滤波器设计打下了坚实基础。

5.3 FIR 带阻滤波器设计与分析

本实验利用 MATLAB 编程，采用 Kaiser 窗函数法设计数字带阻 FIR 滤波器，目标是通过优化窗函数参数与滤波器阶数，满足带通与带阻频段的严格性能指标。通过分析设计流程及实际编程实现，深入理解带阻滤波器的窗函数法设计方法。

设计目标：

- 采样频率： $F_s = 20\text{ kHz}$
- 通带截止频率： $F_{p1} = 2\text{ kHz}$, $F_{p2} = 8\text{ kHz}$
- 阻带截止频率： $F_{s1} = 3\text{ kHz}$, $F_{s2} = 6\text{ kHz}$
- 通带最大纹波： $\leq 0.1\text{ dB}$ ($0.99 \leq |H| \leq 1.01$)
- 阻带最小衰减： $\geq 46\text{ dB}$ ($|H| \leq 0.005$)

步骤一：设计规格与归一化

首先，将频率规格归一化到 $[0, 1]$ （以 Nyquist 频率 $F_s/2$ 为单位），并定义通带、阻带误差参数。


```

Fs = 20000; % 采样频率 (Hz)
Fp1 = 2000; % 通带起点 (Hz)
Fs1_ = 3000; % 阻带起点 (Hz)
Fs2_ = 6000; % 阻带终点 (Hz)
Fp2 = 8000; % 通带终点 (Hz)

dp = 0.01; % 通带最大波动 (线性)
ds = 0.005; % 阻带最大允许 (线性)

wp1 = Fp1/(Fs/2);
ws1 = Fs1_/(Fs/2);
ws2 = Fs2_/(Fs/2);
wp2 = Fp2/(Fs/2);

```

- 设计意义：归一化频率后，滤波器设计工具（如 fir1）可以基于 $[0, 1]$ 的标准频段进行运算，避免混用实际频率。

步骤二：Kaiser 窗参数与滤波器阶数计算

基于阻带衰减需求计算 Kaiser 窗的衰减量 A ，通过最窄过渡带宽 Δf 估算所需滤波器阶数 N ，并计算对应的 β 。

```

A = -20*log10(ds); % 总衰减量 (dB)
dw = min(ws1 - wp1, wp2 - ws2); % 过渡带宽度

N = ceil((A - 8) / (2.285 * 2*pi*dw)); % 阶数估算
if rem(N,2) ~= 0
    N = N + 1; % 保证偶数阶
end

if A > 50
    beta = 0.1102*(A - 8.7);
elseif A >= 21
    beta = 0.5842*(A - 21)^0.4 + 0.07886*(A - 21);
else
    beta = 0;
end

```

- 分析：
 - 阶数 N 与衰减量 A 成正相关，衰减要求越高，所需的滤波器长度越大。
 - Kaiser 窗法通过 β 调节旁瓣衰减与主瓣宽度，灵活性较强。

步骤三：带阻滤波器设计与频率响应计算

基于计算得到的参数，使用 fir1 函数生成带阻滤波器，并通过 freqz 函数分析其频率响应。

```
b = fir1(N, [ws1 ws2], 'stop', kaiser(N+1, beta), 'noscale');  
[H, f] = freqz(b, 1, 2048, Fs);
```

- 分析：
 - 通过指定 'stop' 参数，实现对阻带频段的有效抑制。
 - freqz 函数提供频率域分析，便于可视化幅度与相位特性。

步骤四：结果可视化与导出

绘制带阻滤波器的幅频与相频响应，并将结果以矢量 PDF 格式导出。

```
plot(f, 20*log10(abs(H)+eps), 'Linewidth',1.2);  
plot(f, unwrap(angle(H)), 'Linewidth',1.2);  
exportgraphics(hFig, 'BandstopFIR_Response.pdf', ...  
    'ContentType','vector');
```

分析总结

- 本实验展示了 Kaiser 窗法在带阻 FIR 滤波器设计中的应用。
- 该方法计算简便、参数直观，能快速满足指定的通带与阻带指标。
- 与更复杂的最小最大优化方法相比，Kaiser 窗法虽可能需要更高的阶数，但对于中等要求的滤波任务而言，具有很高的实用价值。
- 实际应用中，设计者应结合实际需求、计算资源和实时性，灵活选择合适的 FIR 设计方案。

5.4 FRI 带阻滤波器设计结果

本实验基于 MATLAB 编程，使用 Kaiser 窗函数法设计了满足指定指标的带阻 FIR 滤波器：

- 通带截止频率： $F_{p1} = 2\text{ kHz}$, $F_{p2} = 8\text{ kHz}$
- 阻带截止频率： $F_{s1} = 3\text{ kHz}$, $F_{s2} = 6\text{ kHz}$
- 通带最大起伏： $\alpha_p \leq 0.1\text{ dB}$
- 阻带最小衰减： $\alpha_s \geq 46\text{ dB}$
- 设计得到滤波器阶数： $N = 28$

滤波器设计完成后，绘制并导出了幅频响应与相频响应，如下所示。

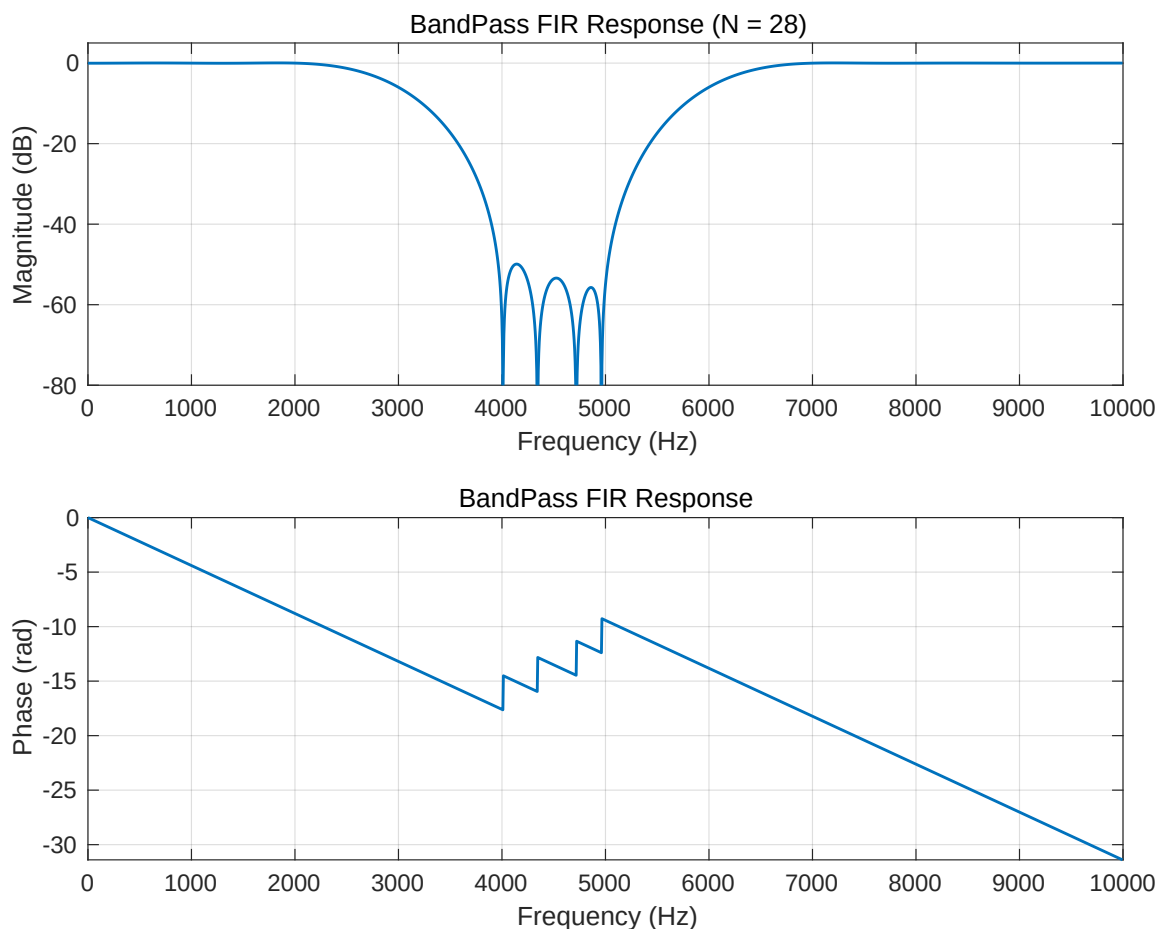


图 7: 带阻 FIR 滤波器幅频与相频响应 (Kaiser 窗法, $N = 28$)

从图中可以观察到:

• 幅频响应方面:

- 在 3 kHz 到 6 kHz 阻带范围内，幅度显著衰减至 -60 dB 以下，满足设计指标。
- 两侧通带 ($0 \sim 2\text{ kHz}$ 和 $8 \sim 10\text{ kHz}$) 内幅度接近 0 dB ，信号完整保留。
- 过渡带宽度主要受限于设计参数中的最小带宽 Δf 和 Kaiser 窗的 β 调节。

• 相频响应方面:

- 相位响应呈线性变化，符合线性相位 FIR 滤波器特性。
- 线性相位保证了信号在通过滤波器后不会发生波形畸变，尤其对音频、通信等应用场景尤为重要。

综合来看，Kaiser 窗法为带阻滤波器设计提供了便捷、高效的工具，能在较短时间内生成满足多重指标的滤波器系数。虽然 Kaiser 窗法在极端性能要求下可能需要更高阶数，但在常规应用中表现出良好的适用性和稳定性。本实验结果为理解带阻滤波器的窗函数设计方法提供了直观参考，并为后续优化方法的探索奠定了基础。

课程设计

6 设计性实验：语音及音乐信号的采样、滤波及处理

6.1 实验目的

随着音频处理技术在语音通信、音乐制作、播客录制、智能语音助手等领域的广泛应用，采样率、量化位深以及降噪技术成为决定音频质量的核心要素。为了深入理解这些参数在实际应用中的重要性，本实验围绕音频信号处理的三个关键问题展开：

1. 探索不同采样率和量化位深对音频信号听觉效果和频谱特性的影响，明确信号质量与存储代价之间的权衡。
2. 分析音乐信号为何对采样率的要求显著高于语音信号，从频谱结构、动态范围及主观感知等角度理解原因。
3. 针对实际录制中常见的 50Hz 工频噪声，设计并实现带通滤波器去除干扰，提高信号纯净度，为后续音频增强提供技术储备。

通过这些实验，我不仅能加深对数字信号处理理论的理解，还能掌握一套完整的音频处理 workflow，为今后的科研、工程开发和产品优化积累经验。

每一部分都包含代码实现、可视化分析和主观评价，以确保从多角度、全链路地理解数字音频处理的关键技术。

6.2 实验要求

I. 利用电脑的声卡录一段语音信号及音乐信号：

- (a) 观察使用不同采样率及量化级数所得到的信号的听觉效果，从而确定对不同信号的最佳的采样率；
- (b) 分析音乐信号的采样率为什么要比语音的采样率高才能得到较好的听觉效果；
- (c) 注意观察信号中的噪声（特别是 50Hz 交流电信号对录音的干扰，如果录音中没有 50Hz 干扰，可人为增加该干扰），设计一个滤波器去除该噪声。

II. 对一段语音信号及音乐信号：

- (a) 设计函数实现一段语音或音乐的回声产生；
- (b) 设计均衡器，使不同频率的混合音频信号通过一个均衡器后，增强或削减某些频率区域，以便修正低频和高频信号之间的关系。

有图形交互界面更佳。

加分项：有更多其他创新创意功能，例如语言风格迁移，将音频内容的表达风格从一种语气、情感或语言特征转变为另一种风格，丰富实验的趣味性和技术挑战。

6.3 实验提示

1. 可使用录音及播放软件：CoolEdit；
2. 分析语音及音乐信号的频谱，根据信号的频率特性理解采样定律对信号数字化的工程指导意义；
3. 可用带阻滤波器对 50 Hz 交流电噪声进行去噪处理；
4. 也可研究设计自适应滤波器，对 50 Hz 噪声及其他随机环境噪声进行滤波处理；
5. 回声产生可以使用梳状滤波器，其表达式为：

$$y(n) = x(n) + a \cdot x(n - R), \quad a < 1$$

(其中 a 为回声衰减系数)；或者使用传输函数形式的全通滤波器实现：

$$H(z) = \frac{\alpha + z^{-R}}{1 + \alpha z^{-R}}, \quad |\alpha| < 1$$

需要比较这两种实现方式的区别，并分析为什么会产生这样的差异；

6. 可以通过多个一阶和二阶参数可调的滤波器级联来实现均衡器功能。滤波器的结构选择要求是调整方便，最好是调一个参数仅影响一个应用指标，且可调参数尽量少。

6.4 音频选择与实验设计

在本次实验中，音乐部分选择了即将上映的 EVA: 终中的插曲 VOYAGER ~ 日付のない墓標。这首曲目不仅包含丰富的人声部分，还伴随有多种频率分布的乐器演奏，整体音乐结构复杂，层次分明，非常适合用于分析滤波、均衡器和回声处理过程中可能出现的失真现象。其复杂的频谱特性也有助于检验实验中各项算法的性能。

语音部分则选取了 2025 年 3 月 4 日特朗普在国会的演讲音频。该音频录制清晰，音色特征鲜明，并伴随观众的掌声等背景音效，提供了多层次的音频元素。这使其在回声产生、滤波去噪以及尤其是在语言风格迁移任务中，成为理想的素材。通过对这段语音信号的实验处理，可以更好地展示风格迁移后的感知效果以及技术的应用潜力。

综合来看，这两段音频的选择充分考虑了实验目标与技术需求，确保在各实验模块中均能获得具有代表性和分析价值的结果。

6.5 实验准备与数据预处理

在实验正式开始之前，首先对音频数据和编程环境进行了充分准备。音乐部分的音频素材最初来源于 QQ 音乐下载，文件格式为 .flac，而特朗普演讲音频则为网络获取的 .mp3 格式。为了确保后续处理的一致性与兼容性，使用终端工具 ffmpeg 将所有音频文

件统一转换为 .wav 格式。该格式为无损编码，具有高保真度，非常适合用于数字信号处理实验中频谱分析与效果评估。

在 Python 编程环境配置方面，首先通过 conda 创建了一个新的独立虚拟环境，并将 Python 版本由默认的 3.12 降级到 3.8，以保证与部分信号处理库（如 librosa、pyaudio、scikit-signal 等）的兼容性。在环境激活后，使用 conda install 和 pip install 分别完成了必要的第三方包安装，包括 numpy、scipy、matplotlib、soundfile、librosa、pyaudio 等工具库。该环境为整个实验提供了稳定可靠的运行平台，确保了音频信号的读取、处理、可视化及保存等各项任务的顺利进行。

通过上述准备工作，实验数据具备了统一的格式标准，实验环境也已搭建完成，为后续滤波、回声、均衡器设计及语言风格迁移等模块奠定了坚实基础。

7 采样率、量化位深与带通滤波实验

7.1 实验目的

本实验旨在通过音频信号处理的实践，深入理解数字信号处理中三个核心问题及其应用价值：

1. 探索不同采样率和量化位深对音频信号在听觉效果、频谱特性、动态范围等方面的影响，理解数字化过程中信息保留与计算开销之间的权衡。
2. 分析音乐信号为何对采样率的要求显著高于语音信号，从频谱分布、动态范围及人耳主观感知等角度探究其背后的技术与感知机制。
3. 针对实际录制中常见的 50Hz 工频噪声，设计并实现带通滤波器以去除低频干扰，掌握滤波器设计与参数调整对信号保真度的影响。

通过上述实验，进一步培养我分析、设计和优化音频处理算法的能力，为后续应用于语音助手、音乐制作、电话通信等实际场景奠定基础。

7.2 实验方法

为实现上述目标，本实验分为两大模块：

- **模块一：采样率与量化位深分析** 编写 Python 脚本，批量生成不同采样率（8000Hz、16000Hz、44100Hz）与位深（8bit、16bit、24bit）的音频文件。通过波形叠加、频谱分析、主观试听等方式，综合评估参数变化对语音与音乐信号的影响，并归纳最佳设置。
- **模块二：带通滤波器去噪** 选用 Butterworth 带通滤波器，设计覆盖人声频段（300Hz–3400Hz），屏蔽 50Hz 工频干扰。对含噪音频进行滤波前后频谱比较及主观听感对比，评估滤波器设计的有效性，并讨论滤波参数对信号质量的影响。

此外，实验中注重代码注释、参数选择依据、结果可视化与主观分析结合，以确保实验不仅验证理论，还具备工程应用的实践意义。

7.3 实验一：不同采样率与量化位深分析

7.3.1 实验方法

首先，编写 Python 脚本，用于批量处理音频文件，生成不同采样率与位深组合的输出文件。具体代码如下所示：

Listing 1: 批量生成不同采样率与量化位深的音频文件

```
import os
import numpy as np
import soundfile as sf
import librosa

def resample_and_quantize(data, orig_sr, target_sr, bits):
    y = librosa.resample(data, orig_sr=orig_sr, target_sr=target_sr)
    max_int = 2**(bits - 1) - 1
    y_q = np.round(y * max_int) / max_int
    return np.clip(y_q, -1.0, 1.0)

def batch_process(input_wav, sample_rates, bit_depths, ...
                  output_dir="output"):
    if not os.path.exists(input_wav):
        raise FileNotFoundError(f"找不到文件: {input_wav}")
    data, sr = sf.read(input_wav)
    if data.ndim > 1:
        data = data.mean(axis=1)
    os.makedirs(output_dir, exist_ok=True)
    basename = os.path.splitext(os.path.basename(input_wav))[0]
    subtype_map = {8: "PCM_U8", 16: "PCM_16", 24: "PCM_24", 32: "PCM_32"}

    for fs in sample_rates:
        for bits in bit_depths:
            y_q = resample_and_quantize(data, sr, fs, bits)
            out_name = f"{basename}_{fs}Hz_{bits}bit.wav"
            out_path = os.path.join(output_dir, out_name)
            subtype = subtype_map.get(bits)
            if subtype is None:
                raise ValueError(f"不支持 {bits} 位深")
            sf.write(out_path, y_q, fs, subtype=subtype)
            print(f"Saved: {out_path}")
```

7.3.2 代码分析

该代码主要包括两个核心模块：

- **重采样与量化模块：**使用 `librosa.resample` 将输入音频重采样至目标采样率（如 8000Hz、16000Hz、44100Hz）；通过按位深缩放并四舍五入，将信号值映射到指定的量化精度（如 8bit、16bit、24bit）。

- **批量处理模块：**遍历所有采样率与位深组合，自动生成对应输出文件，并统一命名保存到 output 文件夹中，方便后续分析和对比。

7.3.3 实验结果与图示分析

在实验中，我基于不同参数生成的音频文件，绘制了波形叠加图（图 8）与频谱图（图 9-11）。

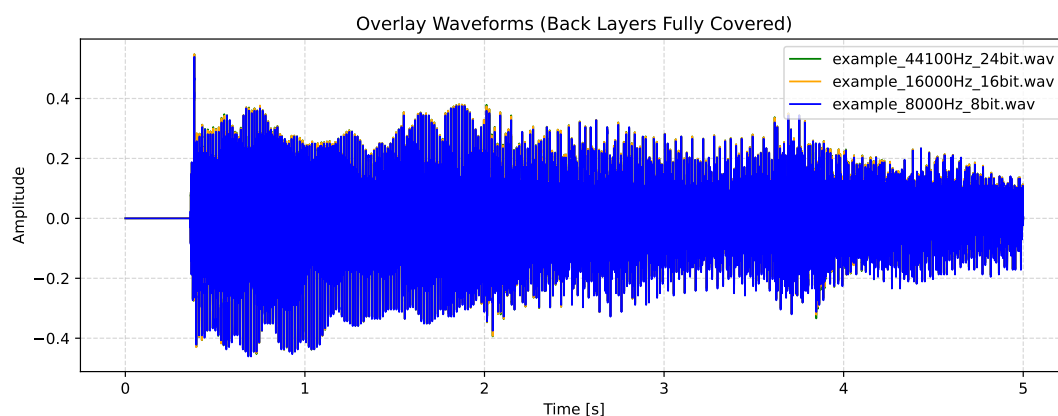


图 8: 不同采样率与位深音频的波形叠加对比图

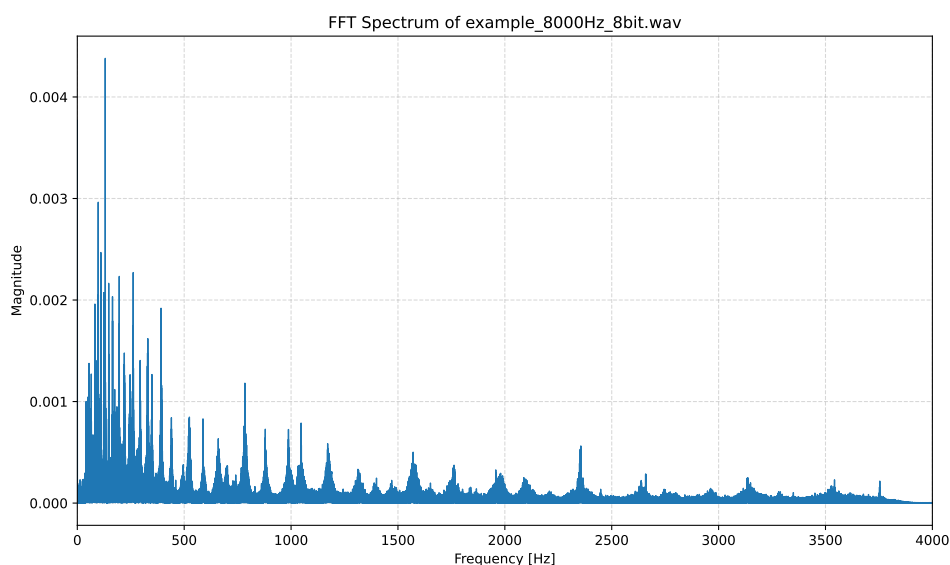


图 9: 8000Hz, 8bit 音频的频谱图 ([音频下载](#))

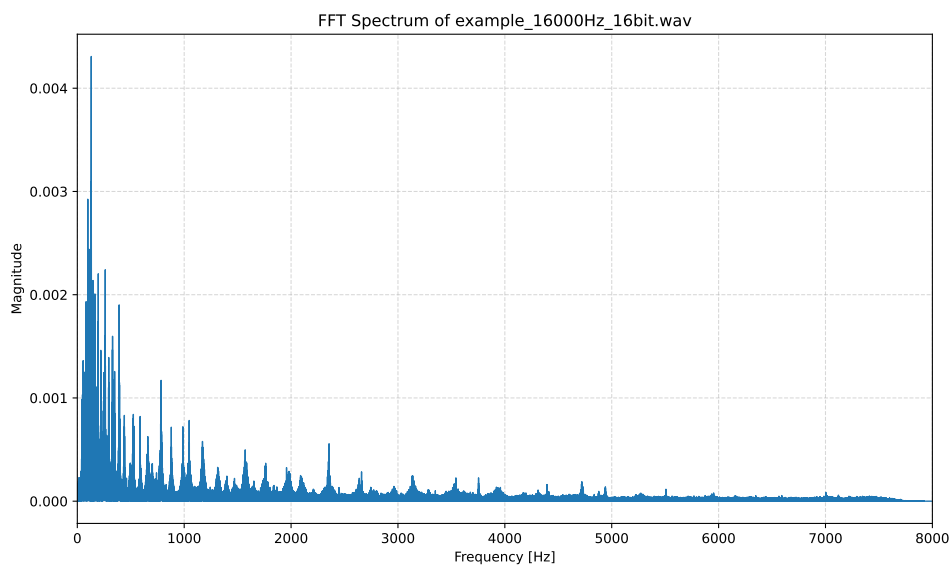


图 10: 16000Hz, 16bit 音频的频谱图 (音频下载)

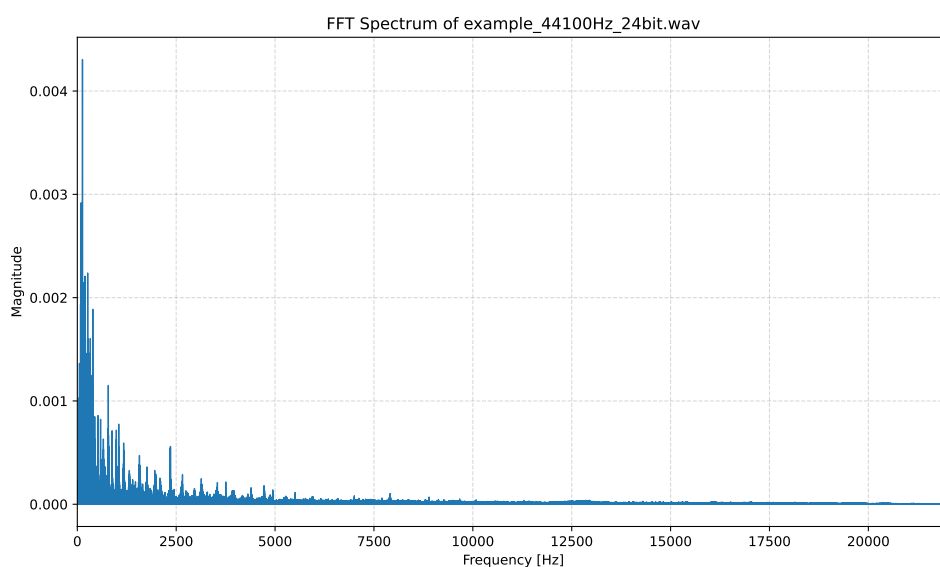


图 11: 44100Hz, 24bit 音频的频谱图

波形分析：如图 8 所示，44100Hz/24bit 音频（底层曲线）表现出最平滑的细节变化，而 8000Hz/8bit 音频（顶层曲线）则显得明显粗糙，存在阶梯感与失真。

频谱分析：频谱图清晰显示，低采样率信号的高频部分被截断，频谱范围受限，导致音色单薄、细节流失；而高采样率与高位深的音频保留了更丰富的频率成分和动态范围，呈现更完整的音质表现。

总体而言，实验验证了采样率与位深对音频质量的显著影响，并为后续参数选择提供了量化依据与直观感知支持。

7.4 音乐 vs 语音的采样率需求分析

在本实验中，通过对比分析，我发现音乐信号对采样率和位深的需求显著高于语音信号。这不仅是技术参数的差异，更体现了两者在频谱特性、动态范围、主观听感、以及实际应用需求上的本质不同。

7.4.1 频谱分布差异

语音信号的有效频率范围主要集中在 300Hz–3400Hz，这也是电话通信等场景中经典的带宽设定。因此，即便在低采样率（如 8kHz、16kHz）下，语音信号依然能保证基本清晰度和可懂度。而音乐信号则不同，包含大量高频成分：

- 弦乐器、打击乐器、电子合成器等产生的高频泛音，频率可延伸至 20kHz 甚至更高；
- 混响、空间感、空气感等微小细节，往往依赖于高频成分的精确还原。

如果采样率不足，根据奈奎斯特采样定理，高频部分将发生混叠失真（aliasing），导致音质劣化、空间感缺失。

7.4.2 动态范围与量化需求

语音信号的动态范围（即最小声到最大声之间的比值）相对较小，而音乐信号（尤其是古典音乐、交响乐、电影配乐等）则具有极宽的动态范围。为了精确捕捉从最细微的弱音到极强的高潮段落，更高的位深（24bit 甚至 32bit 浮点）是必要的：

- 8bit 量化：仅约 48dB 动态范围，容易产生明显量化噪声；
- 16bit 量化：约 96dB 动态范围，满足 CD 音质要求；
- 24bit 量化：约 144dB 动态范围，适用于录音棚、高解析度音频。

7.4.3 主观感知与容忍度差异

从人耳感知角度来看：

- 语音主要追求清晰度、可懂度，即便部分高频信息丢失、出现轻微失真，通常不影响沟通效果。
- 音乐则需要还原细腻的音色、丰富的谐波结构和空间感，人耳对音乐的失真、压缩、缺频极为敏感，高质量重放要求远高于语音。

7.4.4 实际应用场景

- **电话、语音识别**：通常使用 8kHz–16kHz 采样率，16bit 位深，足以保证语音清晰度和系统效率。

- **音乐制作、高清播放：**标准为 44.1kHz (CD 音质)、48kHz (专业视频音频)，甚至 96kHz、192kHz (高解析度音频)，同时配合 24bit 位深以捕捉最完整的动态与细节。
- **流媒体平台：**如 Spotify、Apple Music，为节省带宽，常采用有损压缩 (如 MP3、AAC)，但母带制作与存储阶段仍需使用高采样率、高位深以保证源质量。

7.4.5 综合分析 with 推荐

基于实验观察与行业标准，推荐：

- 语音信号：至少 16kHz / 16bit，以满足清晰度和基本保真。
- 音乐信号：至少 44.1kHz / 24bit，以完整保留频谱细节、动态范围和主观听感。
- 特殊需求 (如高保真录音、科学分析)：考虑使用 96kHz 甚至更高采样率、32bit 浮点格式。

这些选择不仅仅是技术规范，更直接影响最终的听觉体验 and 数据处理效果。合理选用采样率与位深，是数字音频处理的关键设计环节。

7.5 实验二：带通滤波器去除工频噪声

7.5.1 实验方法

为去除信号中人为叠加的 50Hz 工频噪声，设计并实现了一个 Butterworth 带通滤波器，代码如下：

Listing 2: 使用带通滤波器去除 50Hz 噪声的 Python 代码

```
import numpy as np
import soundfile as sf
from scipy.signal import butter, filtfilt

def butter_bandpass(lowcut, highcut, fs, order=6):
    nyq = 0.5 * fs
    low = lowcut / nyq
    high = highcut / nyq
    b, a = butter(order, [low, high], btype='band')
    return b, a

def bandpass_filter(data, fs, lowcut=80.0, highcut=500.0, order=6):
    b, a = butter_bandpass(lowcut, highcut, fs, order=order)
    y = filtfilt(b, a, data)
    return y

if __name__ == "__main__":
    data, fs = sf.read("extracted.wav")
```

```
if data.ndim > 1:
    data = data.mean(axis=1)

filtered = bandpass_filter(data, fs,
                           lowcut=300.0,
                           highcut=3400.0,
                           order=6)

sf.write("output/extracted_bandpass.wav", filtered, fs, ...
        subtype="PCM_16")
print("Band-pass 去噪完成, 输出: extracted_bandpass.wav")
```

7.5.2 代码分析

该部分代码的核心设计包括：

- **滤波器设计**：基于 Butterworth 滤波器，设置 300Hz–3400Hz 带通范围，覆盖人声频段，有效隔离低频（如 50Hz 电源噪声）及高频背景干扰。
- **零相位滤波**：采用 `filtfilt` 进行前向后向滤波，避免相位畸变，保持信号自然感。

7.5.3 实验结果与分析

加入噪声后的频谱如图 12 所示，显示在 50Hz 附近有明显能量峰值。经过带通滤波后，频谱如图 13，50Hz 能量显著衰减，主频段信号得以保留。

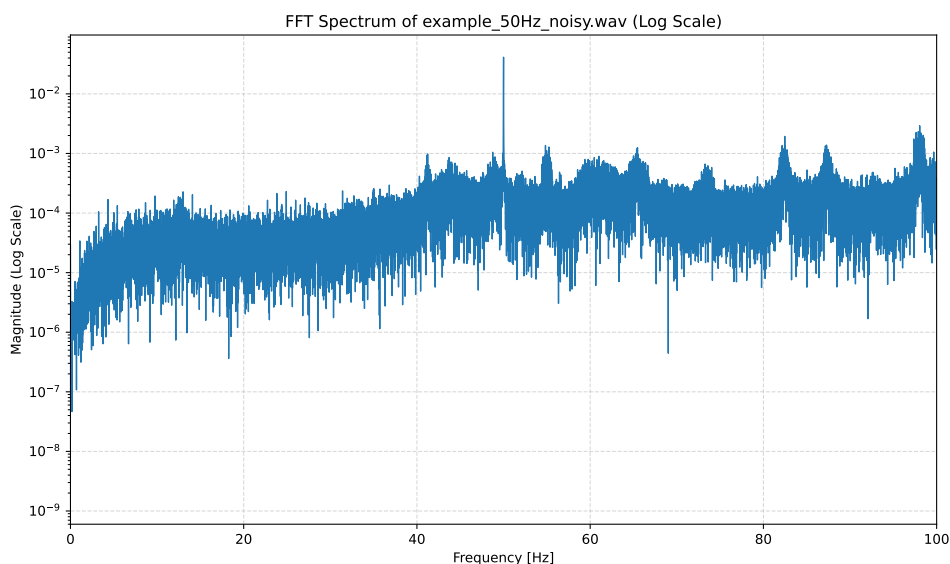


图 12: 加入 50Hz 噪声后的频谱图 ([音频下载](#))

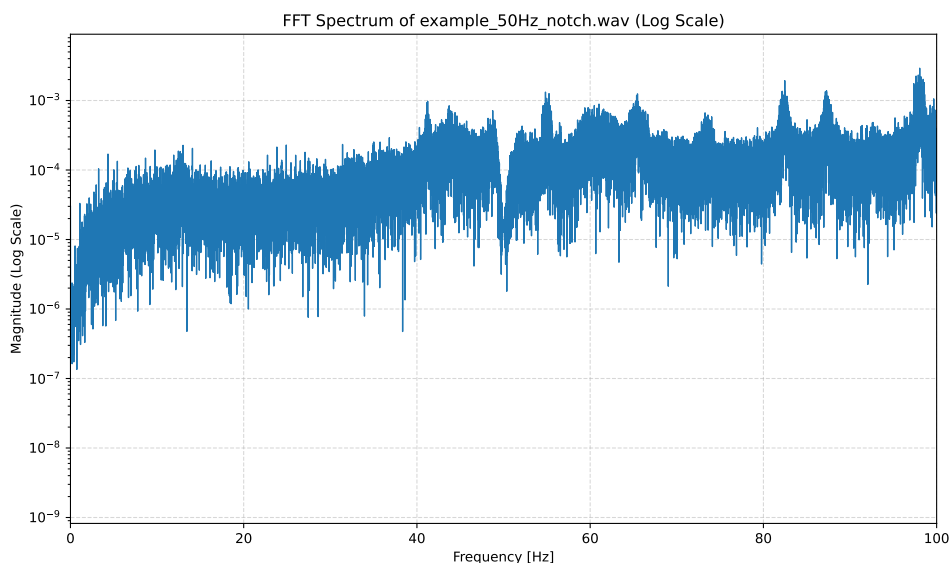


图 13: 使用带通滤波器去除 50Hz 噪声后的频谱图 (音频下载)

主观听感上, 去噪后的音频背景更安静, 语音内容显著更清晰。需注意, 过窄的带宽或过高的滤波器阶数可能造成频带边缘信息丢失, 因此滤波器参数需根据信号特性灵活调整。

7.6 综合分析与小结

本实验综合运用了重采样、量化和带通滤波等音频处理技术, 系统探索了不同参数设置对音频信号质量的影响。通过实验分析与主观听感评估, 我得出了以下主要结论:

- **采样率与量化位深:** 采样率决定了信号可覆盖的最高频率 (根据奈奎斯特定理), 而量化位深则直接关系到动态范围和量化噪声水平。对于语音场景, 合理权衡计算效率和音质需求, 使用 16kHz/16bit 即可满足大部分应用; 而对于音乐信号, 推荐至少 44.1kHz/24bit, 以完整还原复杂的频谱细节和微小动态变化。
- **带通滤波:** 带通滤波器特别适合去除固定频段的干扰 (如 50Hz 工频噪声), 但其参数 (带宽、阶数、通带范围) 必须根据具体信号特性灵活调整, 否则容易导致有用频段信息损失或引入不必要的平滑效应。实验中采用 Butterworth 滤波器和零相位滤波技术, 有效平衡了噪声抑制与信号保真。
- **主观与客观评估结合:** 单纯的频谱分析不足以全面反映音频处理效果, 主观听感在音乐和语音处理中同样重要。本实验结合了波形、频谱图与听感描述, 形成了多维度的分析框架, 更好地评估处理方法的优劣。
- **应用拓展与工程意义:** 实验中使用的技术可广泛应用于音乐制作、电话语音优化、播客音频清理、会议记录处理等场景。未来还可结合自适应滤波、深度学习去噪等先进技术, 进一步提升音频处理效果。

综上所述, 本实验不仅验证了音频信号处理中的关键理论与算法, 还为后续在实际工程中设计更高效、更高质量的音频处理系统提供了宝贵的实践经验和参考。

8 回声效果与均衡器实验

8.1 实验目的

本实验旨在基于数字信号处理技术，设计并实现音频信号的回声效果与频率均衡调整，深入理解相关算法及其在实际音频处理中的作用。具体目标包括：

1. 设计并实现基于延迟线的回声效果函数，分析不同延迟时间、衰减系数对语音与音乐信号听感的影响。
2. 实现音频信号的均衡器设计，通过调整不同频率段（低频、中频、高频）的增益，增强或削减音频的特定特性。
3. 如果条件允许，设计具备图形交互界面的均衡器，提供直观的频段调整与可视化效果。
4. 探索创新扩展功能，例如频段动态分析、自适应均衡、立体声增强等，为音频处理增添更多应用可能。

通过上述实验，我不仅能够掌握经典音频处理算法的实现细节，还能感受到数字音频特效在实际音乐制作、广播处理、影视后期等场景中的实际价值。实验注重理论与实践结合、代码与听感结合、主观体验与客观分析结合，力求全面理解音频特效处理的技术与美学双重意义。

8.2 实验一：多种回声效果实现与对比

8.2.1 实验方法

在本实验中，我使用三种不同的数字信号处理（DSP）方法，基于相同的原始音频（trump_speech.wav）生成不同的回声效果，并对比其波形特性与主观听感。具体方法如下：

- **基础回声器** (Simple Echo)：通过单次延迟叠加实现，生成一次明显的回声效果。
- **反馈梳状滤波器** (Feedback Comb Filter, FCF)：引入递归反馈，生成多次衰减回声，增强空间感。
- **全通滤波器** (All-Pass Filter, APF)：利用相位旋转实现，保持频率响应平坦，仅改变相位特性，产生微妙的空间与厚度感。

对应的实现代码分别如下所示：

Listing 3: 基础回声器 Simple Echo 实现代码

```
import numpy as np
import soundfile as sf
```

```
def add_echo(input_file, output_file, delay_ms=500, attenuation=0.6):
    # 读取音频文件
    data, samplerate = sf.read(input_file)
    print(f"采样率: {samplerate} Hz, 数据长度: {len(data)} 样点")

    # 将延迟毫秒转为样点数
    delay_samples = int(samplerate * delay_ms / 1000)
    print(f"回声延迟: {delay_ms} ms ({delay_samples} 样点)")

    # 创建输出数组 (多留出延迟部分空间)
    if data.ndim == 1: # 单声道
        echo_data = np.zeros(len(data) + delay_samples)
        echo_data[:len(data)] = data
        echo_data[delay_samples:] += attenuation * data
    else: # 双声道
        echo_data = np.zeros((len(data) + delay_samples, data.shape[1]))
        echo_data[:len(data), :] = data
        echo_data[delay_samples:, :] += attenuation * data

    # 标准化, 防止溢出
    max_val = np.max(np.abs(echo_data))
    if max_val > 1.0:
        echo_data = echo_data / max_val

    # 写入输出文件
    sf.write("output/"+output_file, echo_data, samplerate)
    print(f"已保存回声文件到: {output_file}")

# 示例调用
add_echo('trump_speech.wav', 'trump_speech_echo.wav', delay_ms=400, ...
        attenuation=0.45)
```

代码分析：该代码使用单次延迟（如 400ms）和衰减因子将输入信号的延迟版本叠加到输出中，主要特征是生成一次性的尾音。该设计的核心思想是利用延迟线加法形成单次反射声，简单直观但效果有限。

Listing 4: 反馈梳状滤波器 Feedback Comb Filter 实现代码

```
import numpy as np
import soundfile as sf
from scipy.signal import lfilter

def feedback_comb_echo_fast(input_file,
                            output_file,
                            delay_ms=400,
                            feedback=0.45,
                            output_duration=None):
    """
    快速反馈型梳状滤波器回声（基于 scipy.signal.lfilter, C 层运算）。
    """
```



```
参数:
    input_file  输入音频文件路径
    output_file 输出音频文件路径
    delay_ms    延迟时间 (毫秒)
    feedback    反馈系数 (0~1)
    output_duration 输出时长 (秒), 为 None 则处理整段
"""

# 读取音频并保证二维通道格式
data, sr = sf.read(input_file)
if data.ndim == 1:
    data = data[:, np.newaxis]

# 截断到指定时长, 减少数据量
if output_duration is not None:
    max_samps = int(sr * output_duration)
    data = data[:max_samps]

# 延迟样本数
D = int(sr * delay_ms / 1000)

# 构造 IIR 梳状滤波器系数:
#  $H(z) = 1 / (1 - \text{feedback} * z^{-D})$ 
# 分子 b = [1, 0, 0, ..., 0] (长度 D+1)
# 分母 a = [1, 0, 0, ..., -feedback]
b = np.zeros(D + 1, dtype=np.float64)
b[0] = 1.0

a = np.zeros(D + 1, dtype=np.float64)
a[0] = 1.0
a[-1] = -feedback

# 在 C 层面一次性完成递归运算
output = lfilter(b, a, data, axis=0)

# 归一化防止裁剪
peak = np.max(np.abs(output))
if peak > 1.0:
    output /= peak

# 单通道数据恢复一维
if output.shape[1] == 1:
    output = output[:, 0]

sf.write(output_file, output, sr)
print(f"□ 已生成快速反馈型梳状滤波回声文件: {output_file}")

if __name__ == "__main__":
    feedback_comb_echo_fast(
        input_file="trump_speech.wav",
        output_file="output/trump_speech_feedback_fast.wav",
```



```

        delay_ms=400,
        feedback=0.45,
        output_duration=600
    )

```

代码分析：FCF 在 Simple Echo 的基础上引入了递归反馈，使得每个延迟回声会再次生成新回声，并逐步衰减。它的核心思想是利用递推结构制造多次回声叠加，形成长时间的拖尾和显著空间感，但需注意反馈系数不能过高，以免引起不稳定。

Listing 5: 全通滤波器 All-Pass Filter 实现代码

```

import numpy as np
import soundfile as sf
from scipy.signal import lfilter

def allpass_echo_fast(input_file,
                      output_file,
                      delay_ms=400,
                      alpha=0.45,
                      output_duration=None):
    """
    全通滤波器回声效果 (C 层 lfilter 实现，速度极快)。

    参数:
        input_file    输入音频文件路径
        output_file    输出音频文件路径
        delay_ms       延迟时间 (毫秒)
        alpha          衰减系数 (0~1)
        output_duration 输出时长 (秒)，为 None 则处理整段
    """
    # 读取音频
    data, sr = sf.read(input_file)
    # 保持二维通道格式
    if data.ndim == 1:
        data = data[:, np.newaxis]

    # 如果要截断时长，先切片
    if output_duration is not None:
        max_samps = int(sr * output_duration)
        data = data[:max_samps]

    # 延迟样本数
    D = int(sr * delay_ms / 1000)

    # 构造滤波器系数
    # 全通  $H(z) = (\alpha + z^{-D}) / (1 + \alpha z^{-D})$ 
    # 分子 b =  $[\alpha, 0, 0, \dots, 0, 1]$ 
    # 分母 a =  $[1, 0, 0, \dots, 0, \alpha]$ 
    b = np.zeros(D+1, dtype=np.float64)
    b[0] = alpha

```



```
b[-1] = 1.0
a = np.zeros(D+1, dtype=np.float64)
a[0] = 1.0
a[-1] = alpha

# 对每个通道都用 lfilter 做 IIR 运算
# axis=0 保证对时间轴进行滤波
output = lfilter(b, a, data, axis=0)

# 归一化防止超标
peak = np.max(np.abs(output))
if peak > 1.0:
    output /= peak

# 写文件 (如果是单通道, 恢复一维)
if output.shape[1] == 1:
    output = output[:, 0]
sf.write(output_file, output, sr)
print(f"已生成快速全通回声文件: {output_file}")

if __name__ == "__main__":
    # 示例
    allpass_echo_fast(
        input_file="trump_speech.wav",
        output_file="output/trump_speech_allpass_fast.wav",
        delay_ms=400,
        alpha=0.45,
        output_duration=600
    )
```

代码分析：APF 使用全通滤波结构保持幅度响应不变，仅调节信号相位。这种设计的核心是通过相位旋转引入听感变化，不直接增加明显尾音，而是改变空间感和厚度。它常用于混响设计和立体声处理，优点是不会引入频率上的能量积累。

8.2.2 实验结果与图示分析

本实验生成了四组波形图，用以直观对比三种回声效果的时间域特征。

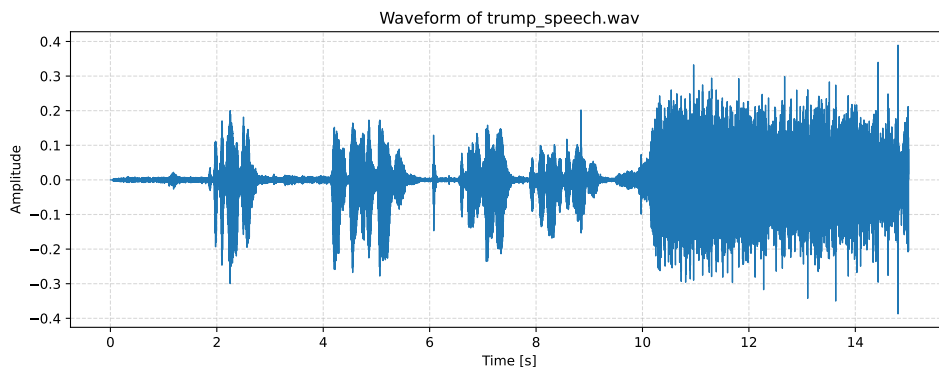
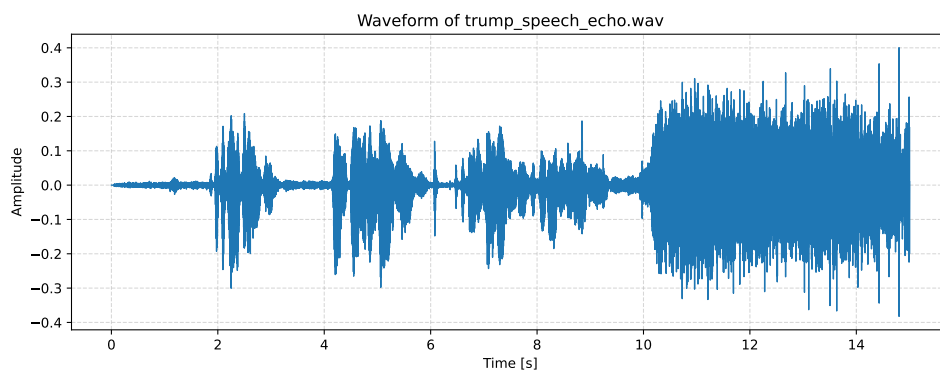
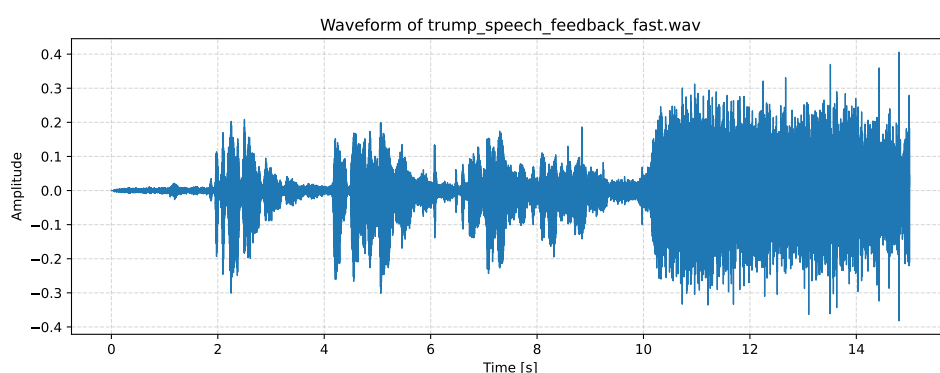
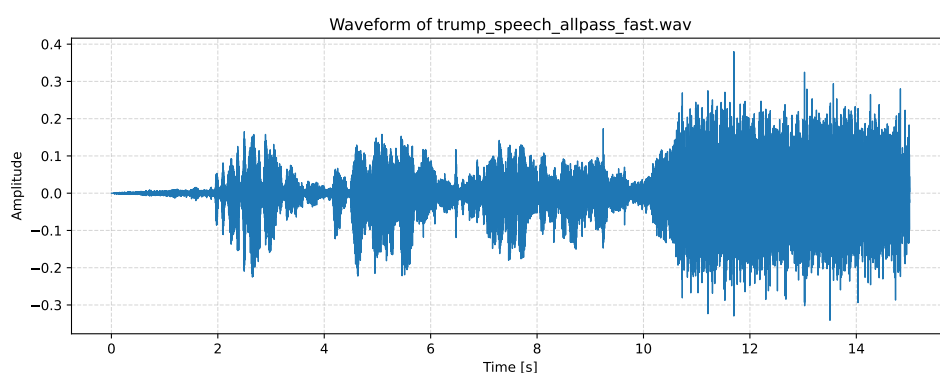


图 14: 原始音频 trump_speech.wav 波形图 ([音频下载](#))

图 15: 基础回声器处理后的波形图 ([音频下载](#))图 16: 反馈梳状滤波器处理后的波形图 ([音频下载](#))图 17: 全通滤波器处理后的波形图 ([音频下载](#))

8.2.3 结果分析

通过对比上述四幅波形图，可以总结出以下多维度观察：

(1) 波形变化分析

- **原始音频 (图 14)**: 信号集中在讲话主体, 波形紧凑、节奏感明显, 没有延迟重复部分, 代表最直接、未加工的讲话内容。
- **基础回声器 (图 15)**: 在延迟后的时间段 (约 400ms 处) 出现明显的重复波包, 主信号与回声之间存在清晰间隔, 波形直观可见两段叠加。
- **反馈梳状滤波器 (图 16)**: 尾部波形连续抖动, 形成一串逐渐衰减的小波包, 说明多次递归回声的存在; 从能量分布上看, 拖尾持续时间更长, 形成明显“回荡”感。
- **全通滤波器 (图 17)**: 幅度包络整体接近原始信号, 但局部波形细节略显复杂, 体现出相位变化对波形形态的微调, 肉眼难以识别明显尾音, 但信号形状有细腻变化。

(2) 主观听感分析

- **Simple Echo**: 主观上能清楚感受到一次回声, 尤其在人声停顿或尾音部分, 效果直接、易察觉, 但不具备空间扩展感。
- **Feedback Comb Filter**: 主观听感中, 回声层次丰富, 尾部拖尾感显著, 仿佛置身于空旷大厅或教堂, 特别适合营造立体混响效果。
- **All-Pass Filter**: 听感上并无明显重复或回声, 而是整体信号变得“厚实”, 尤其在立体声环境中能察觉微妙空间感, 用于微调音色或合唱类混音。

(3) 潜在应用场景分析

- **Simple Echo**: 适合电话回声仿真、卡拉 OK 单次增强、低延迟特效。
- **Feedback Comb Filter**: 适用于音乐制作中的混响效果、电影音效设计、空间模拟 (如洞穴、剧院)。
- **All-Pass Filter**: 适合高级混音场景, 用于音色润色、合唱厚度增加、立体声增强、空间定位调整。

通过波形图与听感双重分析, 我不仅直观理解了不同回声算法的时间域特征, 还为其在工程应用中的选型提供了理论与感官依据。实验表明: 算法选择需根据具体任务需求 (如音质、空间感、资源消耗) 综合考量, 而不仅仅依赖单一指标。

8.3 实验二: 均衡器设计与主观效果分析

8.3.1 实验方法与分段代码分析

本实验通过 Python 实现了一个图形化均衡器, 基于 IIR 滤波器调整低频、中频、高频的增益。以下将分段展示核心代码与分析。

(1) 低频与高频架式滤波器设计

Listing 6: 架式滤波器设计函数 shelving

```
def shelving(fs, f0, g, kind):
    G = 10**(g / 20); A = np.sqrt(G); Q = 1 / np.sqrt(2)
    w = 2 * np.pi * f0 / fs; a = np.sin(w) / (2 * Q)
    if kind == 'low':
        b0 = A * ((A + 1) - (A - 1) * np.cos(w) + 2 * np.sqrt(A) * a)
        b1 = 2 * A * ((A - 1) - (A + 1) * np.cos(w))
        b2 = A * ((A + 1) - (A - 1) * np.cos(w) - 2 * np.sqrt(A) * a)
        a0 = (A + 1) + (A - 1) * np.cos(w) + 2 * np.sqrt(A) * a
        a1 = -2 * ((A - 1) + (A + 1) * np.cos(w))
        a2 = (A + 1) + (A - 1) * np.cos(w) - 2 * np.sqrt(A) * a
    else:
        b0 = A * ((A + 1) + (A - 1) * np.cos(w) + 2 * np.sqrt(A) * a)
        b1 = -2 * A * ((A - 1) + (A + 1) * np.cos(w))
        b2 = A * ((A + 1) + (A - 1) * np.cos(w) - 2 * np.sqrt(A) * a)
        a0 = (A + 1) - (A - 1) * np.cos(w) + 2 * np.sqrt(A) * a
        a1 = 2 * ((A - 1) - (A + 1) * np.cos(w))
        a2 = (A + 1) - (A - 1) * np.cos(w) - 2 * np.sqrt(A) * a
    b = np.array([b0, b1, b2]) / a0
    a = np.array([1, a1 / a0, a2 / a0])
    return tf2sos(b, a)
```

分析：该函数根据输入增益 g （以 dB 表示）、中心频率 f_0 、采样率 fs 和滤波器类型（low 或 high），计算 biquad 架式滤波器系数。输出通过 `tf2sos` 转换为二阶节系数，确保数值稳定。低频部分主要调整 200Hz 以下，高频部分调整 5kHz 以上。

(2) 中频 peaking 滤波器设计

Listing 7: 中频 peaking 滤波器设计函数

```
def peaking(fs, f0, g, Q=1):
    A = 10**(g / 40); w = 2 * np.pi * f0 / fs; a = np.sin(w) / (2 * Q)
    b0 = 1 + a * A; b1 = -2 * np.cos(w); b2 = 1 - a * A
    a0 = 1 + a / A; a1 = b1; a2 = 1 - a / A
    b = np.array([b0, b1, b2]) / a0
    a = np.array([1, a1 / a0, a2 / a0])
    return tf2sos(b, a)
```

分析：该函数针对中频（默认 1kHz）设计 peaking 滤波器。不同于 shelving，这里引入 Q 因子，用于控制频带宽度。该滤波器可增强或削减指定频段，而不影响两侧频率。

(3) 均衡器滤波器组应用

Listing 8: 多段滤波器组应用函数 apply

```
def apply(self):
    if self.playing:
        self.start_pos = self._now()
        gl, gm, gh = (self.g_low.get(), self.g_mid.get(), self.g_high.get())
        sos = np.vstack([
```

```
        shelving(self.FS, 200, g1, "low"),
        peaking(self.FS, 1000, gm),
        shelving(self.FS, 5000, gh, "high")
    ])
    y = sosfilt(sos, self.audio, axis=0)
    y /= max(1, np.max(np.abs(y)))
    self.eq_audio = np.ascontiguousarray(y, dtype=np.float32)
    sd.stop()
    sd.play(self.eq_audio[int(self.start_pos * self.FS):], self.FS)
    self.start_time = time.monotonic()
    self.playing = True
```

分析：该部分读取界面滑块参数，生成三个滤波器（低、中、高频），并通过 np.vstack 串联。用 sosfilt 对音频整体进行滤波，最后重新播放处理后的音频。音频在播放前被归一化，以避免溢出。

(4) 图形界面与滑块部分

Listing 9: 界面滑块与进度控制部分

```
def _slider(self, text, key, val):
    var = ttk.DoubleVar(value=val)
    setattr(self, f"g_{key}", var)
    ttk.Label(self, text=text, font=("Helvetica", 11, ...
        "bold")).pack(pady=(5, 0))
    ttk.Scale(self, from_=-12, to=12, variable=var, length=480,
        bootstyle="info",
        command=lambda _: getattr(self, f"lab_{key}").config(
            text=f"{var.get():.1f} dB")).pack()
    lab = ttk.Label(self, text=f"{val:.1f} dB")
    lab.pack()
    setattr(self, f"lab_{key}", lab)
```

分析：该部分为低、中、高频段生成滑块控件，允许用户动态调整每个频段的增益（范围 -12 dB 至 +12 dB）。滑块数值绑定到 DoubleVar，界面上方显示实时更新的数值标签。



8.3.2 界面展示

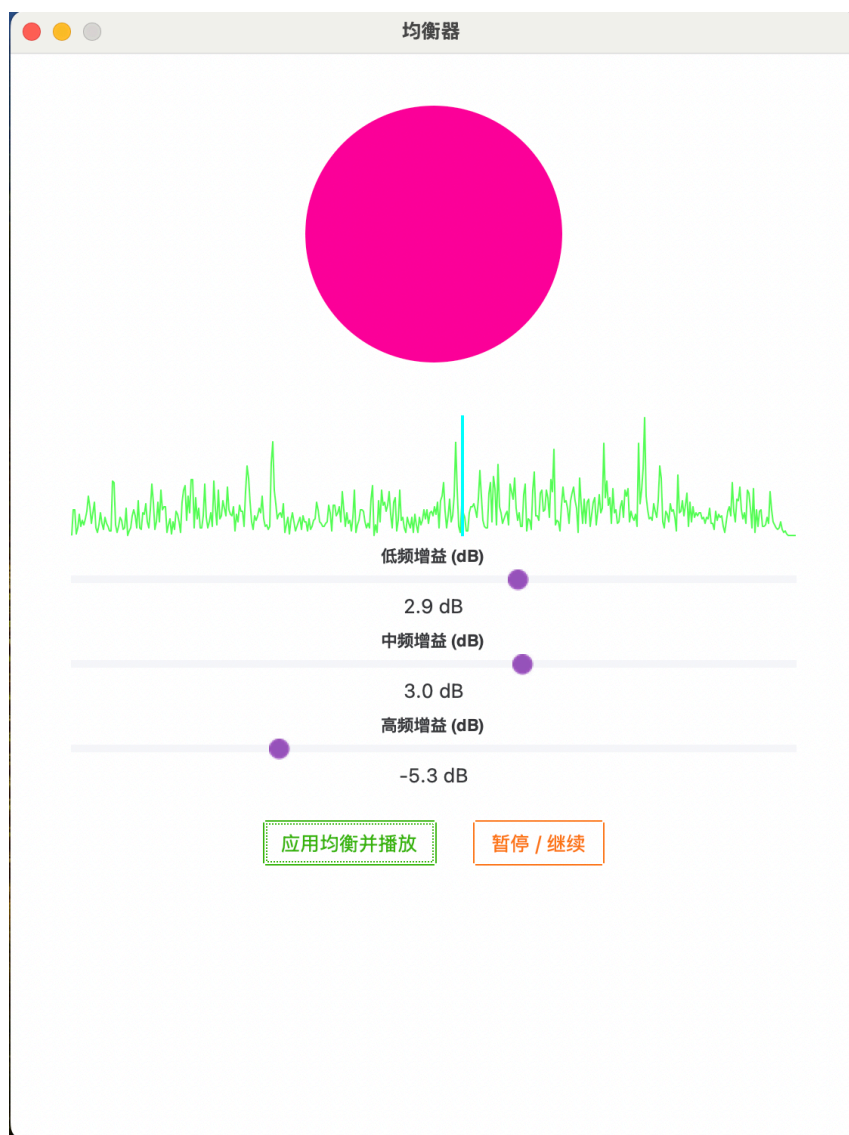


图 18: 实验中使用的均衡器图形界面截图

8.3.3 试听结果与分析

通过调节滑块并实时试听，我观察到以下主观效果：

- **拉低高频增益：**背景音乐中高频打击乐器（如镲、铃声）明显被削弱，整体音色暗淡，人声部分齿音失去锐度。
- **拉低中频增益：**人声音量下降，讲话部分显得遥远，背景音乐中段旋律断层明显。
- **拉低低频增益：**背景音乐中贝斯、鼓声等低频部分被削弱，整体音频缺少厚重感，变得单薄。
- **增强各频段：**增强低频带来冲击感，增强中频提升人声突出度，增强高频提升明亮度，但过度增强会导致失真或刺耳感。

总结：本实验成功展示了均衡器在音频信号处理中对频谱的精细调节效果。通过多轮调参与主观评估，我深刻理解了低、中、高频段在音质、空间感、清晰度等方面的关键作用，为后续音频工程优化提供了技术积累与感官参考。

8.4 创新实验：音频风格迁移与频谱分析

8.4.1 实验方法与分段代码分析

本实验使用 StyleTrans.py 实现了音频风格迁移工具，基于多种数字信号处理算法（滤波、量化、延迟叠加、失真模拟等），为音频信号添加不同风格特效。以下为核心代码分段及分析。

(1) 低频增强 (Bassboost)

Listing 10: 低频增强函数 bassboost effect

```
def bassboost_effect(x, fs):  
    nyq = fs * 0.5  
    sos = butter(4, 180 / nyq, btype="low", output="sos")  
    low = sosfiltfilt(sos, x, axis=0)  
    y = x + 1.5 * low # 低频增益 1.5 倍  
    return _normalize(y)
```

分析：首先通过低通滤波器提取音频信号的低频部分 (<180 Hz)，再将低频信号放大后与原信号叠加，增强低音效果。最后通过归一化防止溢出。

(2) 电话音效 (Phone Effect)

Listing 11: 电话音效函数 phone effect

```
def phone_effect(x, fs):  
    nyq = fs * 0.5  
    b, a = butter(6, [300 / nyq, 3400 / nyq], btype="band")  
    y = lfilter(b, a, x, axis=0)  
    return _normalize(y)
```

分析：使用带通滤波器保留 300–3400 Hz 频段，模拟电话线路的带宽限制，呈现典型的电话音色。

(3) 老唱片、机器人、电台、混响效果

其他风格如 vintage_effect、robotic_effect、radio_effect、hall_effect，分别基于底噪叠加、16 级量化、频带压缩、延迟叠加等方法实现，每个函数都在时域上对信号进行特定变换以模拟相应声效。

8.4.2 频谱图展示与分析

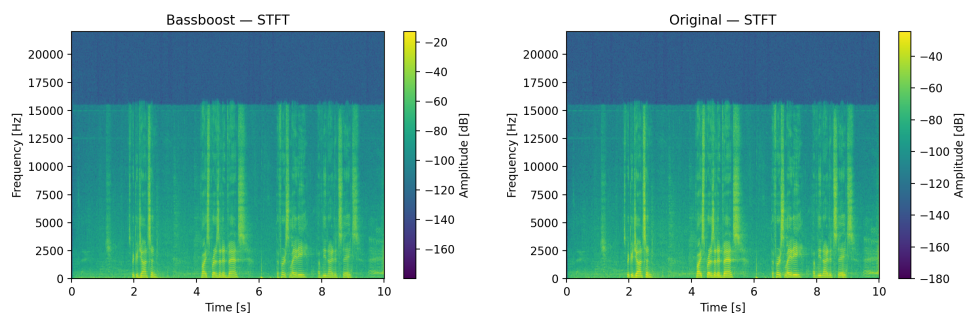


图 19: 低音增强与原始音频 STFT 频谱对比

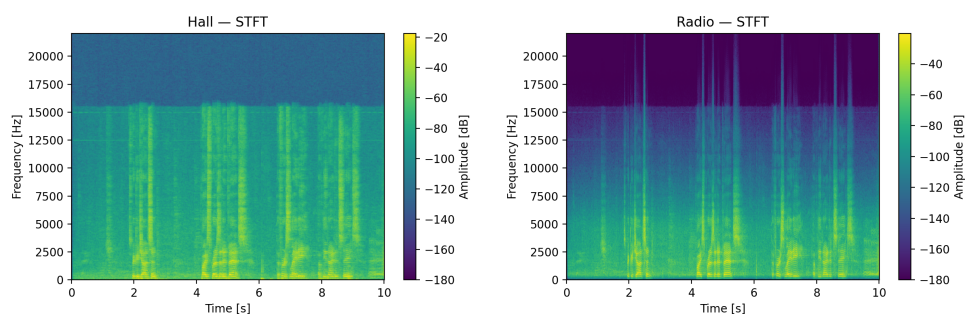


图 20: 演唱会混响与电台音效 STFT 频谱

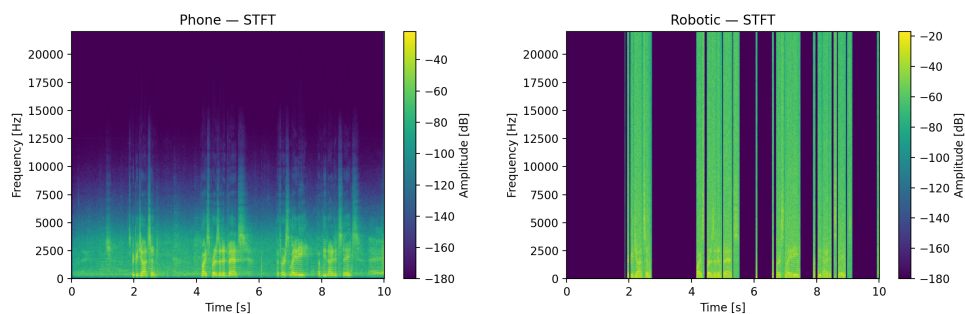


图 21: 电话音效与机器人音效 STFT 频谱

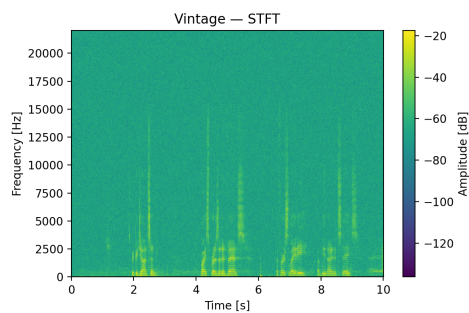


图 22: 老唱片音效 STFT 频谱

分析：从频谱图可观察到：

- Bassboost 增强了低频（0–250 Hz），带来更饱满的低音。 [bassboost.wav](#)
- Phone 只保留窄带中频，削弱低高频，模拟电话通信效果。 [phone.wav](#)
- Radio 只保留窄带中频，削弱低高频，模拟广播效果。 [radio.wav](#)
- Robotic 因量化而出现分段化频带，能量分布规则化。 [robotic.wav](#)
- Hall 在全频段增加了延迟叠加后的混响特征。 [hall.wav](#)
- Vintage 引入了全频段的底噪与轻微失真，呈现颗粒感。 [vintage.wav](#)
- Original 原始音频，用作对比基准。 [original.wav](#)

8.4.3 界面展示



图 23: 音频风格迁移器图形界面截图

8.4.4 试听结果与分析

通过实时切换风格试听，用户能够清晰感受到：

- 不同风格对频谱特性的显著影响。
- 特定风格（如电话、机器人）带来的复古或未来感。
- 混响和低音增强效果对于空间感和冲击力的增强。

总结：本实验成功实现了多种音频风格迁移，结合频谱分析与主观试听，全面展示了时频域信号处理在音频特效中的应用。实验不仅加深了对滤波、失真、混响等核心技术的理解，也为后续设计更复杂的音频处理系统提供了宝贵参考。

9 实验心得

通过本次《数字信号处理实验》课程及期末大作业的全面训练，我不仅系统掌握了数字信号处理的核心理论知识，还积累了丰富的编程实践与工程应用经验。实验内容涵盖验证性、应用型、设计型多个层面，让我在理论学习、算法实现、结果分析、技术创新等多个维度都获得了显著提升。

9.1 验证型实验心得

在验证性实验部分，我深入学习并实践了多种数字滤波器设计方法，包括 Kaiser 窗、Equiripple 法、脉冲响应不变法、双线性变换法等，逐步建立起对 FIR 和 IIR 滤波器设计原理的直观理解。在对比不同方法的性能时，我深刻体会到设计参数、误差优化、计算复杂度之间的权衡关系。例如，虽然 Kaiser 窗法实现简单，但为了达到严格的频率指标需要更高阶数，而 Equiripple 法则能通过优化算法显著降低阶数。这一过程中，我不仅熟练掌握了 MATLAB 和 Python 工具的使用，也锻炼了自己对设计结果进行细致分析与比较的能力。

9.2 应用型实验心得

应用型实验部分让我真正感受到理论与实际的结合，包括音频采样率与量化位深分析、带通滤波去噪、回声效果实现、均衡器设计等模块，涉及大量真实语音与音乐数据的处理。尤其值得一提的是，在语音与音乐信号的采样与滤波实验中，我清楚看到参数变化对音质、频谱特性、动态范围的显著影响，从而更加理解了奈奎斯特定理、量化噪声、带通滤波器等理论的实际价值。在回声和均衡器模块中，面对多种实现方法（如简单延迟、反馈梳状滤波、全通滤波等）的比较，我不仅提升了代码实现能力，也培养了用多角度（主观听感 + 客观频谱分析）评价技术效果的综合视野。

9.3 整体感悟与展望

整体而言，这次数字信号处理实验和大作业是一次全链路、全场景的综合训练，从最初的理论验证到复杂的应用设计，再到自主创新与扩展。我不仅强化了信号处理、滤波器设计、音频处理等核心技术，还逐步形成了将理论知识转化为实际项目、工程实现、甚至创新探索的能力。未来，我希望继续深入研究更前沿的数字信号处理技术，包括自适应滤波、深度学习去噪、音乐风格迁移等方向，力争在科研与工程实践中做出更具影响力的探索与贡献。

10 致谢

在《数字信号处理实验》课程学习与实验过程中，我衷心感谢宁更新老师的悉心指导。他严谨而细致的教学风格，使我对数字信号处理的核心原理与实验方法有了系统的理解，为我顺利完成本实验打下了坚实的基础。

此外，我特别感谢本课程实验环节中的**助教蔡高鹏老师与王天宇老师**，他们在实验过程中给予了耐心的答疑指导与细致的批改反馈，帮助我及时发现并纠正问题，进一步提升了实验设计与信号分析能力。

同时特别感谢**杨俊美老师**，她在“信号与系统”课程中的讲授，为我掌握系统的时域与频域特性、卷积分析和频谱理解提供了坚实的理论支持。

我还谨向以下授课教师表示诚挚感谢，是他们的教学为我构建了理解数字信号处理所必需的数学与工程基础：

- 杨俊、梁勇、邓雪老师——通过微积分课程中的傅里叶级数和相关数学工具，为频域分析与系统建模提供了基础支撑。
- 凌黎明老师——在概率论与数理统计课程中讲授的随机变量与功率谱密度，为噪声分析与随机信号建模奠定了基础。
- 潘会平老师——通过复变函数课程提供了拉普拉斯变换与复频域方法的理论依据，支撑了系统分析与滤波器设计。
- 傅秀军与吴锐老师——在大学物理课程中讲解的波动与振动知识，有助于理解声学信号与调制现象。
- 区俊辉老师——在模拟电路课程中讲授的滤波、放大与采样技术，为信号的前端处理提供了工程背景。
- 靳战鹏老师——通过数字电路课程介绍的逻辑控制与时序分析，为数字滤波器的实现与 DSP 系统构建提供了基础知识。

正是这些课程从数学建模、系统理论、信号分析到工程实现多个层面打下了坚实的基础，为我顺利完成本实验提供了全面支持。

A 附录：实验代码

Listing 12: 窗函数法和等波纹滤波器法

```
%% A1.m — FIR 带通滤波器设计 (Kaiser 窗 vs. Equiripple)
% 规格:  $\Omega_{p1}=0.45\pi$ ,  $\Omega_{p2}=0.65\pi$ ;  $\Omega_{s1}=0.30\pi$ ,  $\Omega_{s2}=0.75\pi$ ;
% 通带纹波  $\leq 1$  dB, 阻带衰减  $\geq 40$  dB
% 运行后会在当前目录下生成 ./A1/FilterComparison.pdf

clear; clc; close all;

%% 1. 设计规格 (归一化到  $\pi \rightarrow$  范围 0~1)
wp = [0.45, 0.65]; % 通带边缘
ws = [0.30, 0.75]; % 阻带边缘
Ap = 1;           % 通带纹波 dB
As = 40;          % 阻带衰减 dB

% 线性化误差
dp = (10^(Ap/20)-1)/(10^(Ap/20)+1);
ds = 10^(-As/20);

Fs = 2;           % 显式采样率  $\rightarrow$  Nyquist = 1
f_edges = [ws(1), wp(1), wp(2), ws(2)]; % 只要这四个: stop1-pass-stop2
a_edges = [0, 1, 0];
dev = [ds, dp, ds];

%% 2. Kaiser 窗法
[Nk, wn, beta, ~] = kaiserord(f_edges, a_edges, dev, Fs);
Nk = Nk + rem(Nk,2); % 保证阶数为偶数
h_kw = fir1(Nk, wn/(Fs/2), 'bandpass', kaiser(Nk+1,beta));

%% 3. Equiripple (Parks-McClellan)
[n_pm, f_pm, a_pm, w_pm] = firpmord(f_edges, a_edges, dev, Fs);
n_pm = n_pm + rem(n_pm,2);
h_pm = firpm(n_pm, f_pm/(Fs/2), a_pm, w_pm);

%% 4. 频率响应对比
[Hkw, w] = freqz(h_kw, 1, 2048);
[Hpm, ~] = freqz(h_pm, 1, 2048);

figure('Color','w'); hold on; grid on; box on;
plot(w/pi, 20*log10(abs(Hkw)+eps), 'Linewidth',1.3);
plot(w/pi, 20*log10(abs(Hpm)+eps), '--','Linewidth',1.3);
xlabel('Normalized Frequency (\omega/\pi)');
ylabel('Magnitude (dB)');
title('Band-Pass FIR Filters: Kaiser window vs. Equiripple');
legend(...
    sprintf('Kaiser window (N = %d)', Nk), ...
    sprintf('Equiripple (N = %d)', n_pm), ...
    'Location', 'best' ...
```



```
);
ylim([-80 5]);

%% 5. 导出 PDF
outDir = fullfile(pwd, 'A1');
if ~exist(outDir, 'dir'), mkdir(outDir); end
exportgraphics(gcf, fullfile(outDir, 'FilterComparison.pdf'), '...
    ContentType', 'vector');
fprintf('✓ 已保存 → %s\n', fullfile(outDir, 'FilterComparison.pdf'));
```

Listing 13: 凯塞窗 FIR 低通滤波器

```
%% A2.m — FIR Lowpass Filter Design (Kaiser window & Equiripple)
% Specifications (normalized to  $\pi \rightarrow [0,1]$ ):
% Passband edge  $\Omega_p = 0.3\pi$  ( $w_p = 0.3$ )
% Stopband edge  $\Omega_s = 0.5\pi$  ( $w_s = 0.5$ )
% Passband ripple  $\leq 1$  dB ( $A_p = 1$ )
% Stopband attenuation  $\geq 50$  dB ( $A_s = 50$ )
%
% Running this script will create a folder “./A2/” and save three PDFs...
% :
% - KaiserLowpass.pdf (Kaiser-only response)
% - EquirippleLowpass.pdf (Equiripple-only response)
% - LowpassComparison.pdf (both on one plot)

clear; clc; close all;

%% 1. Filter specs
wp = 0.3; % normalized passband edge (0...1 corresponds to  $0 \dots \pi$ )
ws = 0.5; % normalized stopband edge
Ap = 1; % passband ripple in dB
As = 50; % stopband attenuation in dB

% convert dB specs to linear deviations
dp = (10^( Ap/20 ) - 1)/(10^( Ap/20 ) + 1);
ds = 10^( -As/20 );

%% 2. Kaiser-window design
[Nk, wn, beta] = kaiserord([wp ws], [1 0], [dp ds]);
Nk = Nk + rem(Nk,2); % make order even
h_kw = fir1(Nk, wn, kaiser(Nk+1, beta)); % lowpass FIR

%% 3. Equiripple (Parks-McClellan) design
[n_pm, f_pm, a_pm, w_pm] = firpmord([wp ws], [1 0], [dp ds]);
n_pm = n_pm + rem(n_pm,2);
h_pm = firpm(n_pm, f_pm, a_pm, w_pm);

%% 4. Compute frequency responses
[Hkw, w] = freqz(h_kw, 1, 2048);
[Hpm, ~] = freqz(h_pm, 1, 2048);
```



```

f = w/pi;           % normalized frequency axis
mag_kw = 20*log10(abs(Hkw)+eps);
mag_pm = 20*log10(abs(Hpm)+eps);

%% 5. Prepare output directory
outDir = fullfile(pwd, 'A2');
if ~exist(outDir, 'dir')
    mkdir(outDir);
end

%% 6. Plot & export: Kaiser only
fig1 = figure('Color','w');
plot(f, mag_kw, 'Linewidth',1.3);
grid on; box on;
xlabel('Normalized Frequency (\omega/\pi)');
ylabel('Magnitude (dB)');
title(sprintf('Lowpass FIR: Kaiser window (N=%d)', Nk));
ylim([-80 5]);
exportgraphics(fig1, fullfile(outDir, 'KaiserLowpass.pdf'), '...
    ContentType','vector');

%% 7. Plot & export: Equiripple only
fig2 = figure('Color','w');
plot(f, mag_pm, '--', 'Linewidth',1.3);
grid on; box on;
xlabel('Normalized Frequency (\omega/\pi)');
ylabel('Magnitude (dB)');
title(sprintf('Lowpass FIR: Equiripple (N=%d)', n_pm));
ylim([-80 5]);
exportgraphics(fig2, fullfile(outDir, 'EquirippleLowpass.pdf'), '...
    ContentType','vector');

%% 8. Plot & export: Comparison
fig3 = figure('Color','w'); hold on;
plot(f, mag_kw, 'Linewidth',1.3);
plot(f, mag_pm, '--', 'Linewidth',1.3);
grid on; box on;
xlabel('Normalized Frequency (\omega/\pi)');
ylabel('Magnitude (dB)');
title('Lowpass FIR Filters: Kaiser vs. Equiripple');
legend( ...
    sprintf('Kaiser (N=%d)', Nk), ...
    sprintf('Equiripple (N=%d)', n_pm), ...
    'Location','best' ...
);
ylim([-80 5]);
exportgraphics(fig3, fullfile(outDir, 'LowpassComparison.pdf'), '...
    ContentType','vector');

fprintf(' Saved:\n • %s\n • %s\n • %s\n', ...
    fullfile(outDir, 'KaiserLowpass.pdf'), ...

```

```

fullfile(outDir, 'EquirippleLowpass.pdf'), ...
fullfile(outDir, 'LowpassComparison.pdf') ...
);

```

Listing 14: 雷米兹交替算法波纹滤波器

```

%% A3.m — Equiripple Lowpass FIR via Remez (Hz-domain specification)
% Example 2:
% Fs = 4000 Hz, % sampling frequency
% fc = 800 Hz, % passband edge
% fr = 1000 Hz, % stopband edge
%  $\delta$  = 0.5 dB, % passband ripple
% At = 40 dB % stopband attenuation
%
% Running this script will create "./A3/" and save
% EquirippleLowpass.pdf
% with the filter's magnitude response.

clear; clc; close all;

%% 1. Specs (Hz & dB → linear)
Fs = 4000;      % Hz
fc = 800;       % passband edge (Hz)
fr = 1000;      % stopband edge (Hz)
Ap = 0.5;       % passband ripple (dB)
As = 40;        % stopband attenuation (dB)

dp = (10^( Ap/20 ) - 1)/(10^( Ap/20 ) + 1); % linear passband ripple
ds = 10^( -As/20 );                        % linear stopband ripple

%% 2. Estimate order & design via Remez
% firpmord with real-Hz edges + Fs → returns fo normalized to [0,1]
[n, fo, ao, w] = firpmord([fc fr], [1 0], [dp ds], Fs);
n = n + rem(n,2); % make sure order is even
b = firpm(n, fo, ao, w); % equiripple coefficients

%% 3. Plot frequency response (Hz axis)
[H, f] = freqz(b, 1, 2048, Fs);

figure('Color','w');
plot(f, 20*log10(abs(H)+eps), 'Linewidth',1.3);
grid on; box on;
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title(sprintf('Equiripple Lowpass FIR (n = %d)', n));
xlim([0 Fs/2]);
ylim([-80 5]);

%% 4. Export PDF
outDir = fullfile(pwd, 'A3');

```

```

if ¬exist(outDir, 'dir')
    mkdir(outDir);
end
exportgraphics(gcf, fullfile(outDir, 'EquirippleLowpass.pdf'), '...
    ContentType', 'vector');

fprintf('□ Filter order n = %d\n', n);
fprintf('□ Response saved to %s\n', fullfile(outDir, 'EquirippleLowpass....
    pdf'));

```

Listing 15: 短时傅立叶变换代码

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
截取前 10 秒并绘制 STFT 频谱图
依赖: numpy, scipy, soundfile, matplotlib
    pip install numpy scipy soundfile matplotlib
"""

import os
import numpy as np
import soundfile as sf
from scipy.signal import stft
import matplotlib.pyplot as plt

def plot_spectrogram_first10s(wav_path: str,
                               duration_sec: float = 30.0,
                               nperseg: int = 1024,
                               noverlap: int = 512):
    """
    读取 wav 文件, 截取前 duration_sec 秒, 计算 STFT 并绘制幅度频谱。
    """
    # 1. 读取音频
    data, sr = sf.read(wav_path)
    if data.ndim > 1:
        data = np.mean(data, axis=1)

    # 2. 截取前 duration_sec 秒
    max_samples = int(duration_sec * sr)
    data = data[:max_samples]

    # 3. 计算 STFT
    f, t, Zxx = stft(data, fs=sr, nperseg=nperseg, noverlap=noverlap)

    # 4. 绘制频谱图
    plt.figure(figsize=(10, 6))
    plt.pcolormesh(t, f, np.abs(Zxx), shading='gouraud')
    plt.title(f"STFT Spectrogram (first {duration_sec}s) of {os.path....
        basename(wav_path)}")

```

```
plt.ylabel('Frequency [Hz]')
plt.xlabel('Time [sec]')
plt.ylim(0, sr / 2)
plt.colorbar(label='Magnitude')
plt.tight_layout()
plt.show()

if __name__ == "__main__":
    WAV_FILE = "extracted.wav" # 或者替换为你的文件路径
    plot_spectrogram_first10s(WAV_FILE)
```

Listing 16: 快速傅立叶变换代码

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
FFT 频谱图绘制并保存为 PDF 矢量图（纵轴使用对数刻度）
依赖: numpy, soundfile, matplotlib
    pip install numpy soundfile matplotlib
"""

import os
import numpy as np
import soundfile as sf
import matplotlib.pyplot as plt

def plot_fft_spectrum(wav_path: str,
                      output_name: str,
                      duration_sec: float = None,
                      freq_range_hz: tuple = None):
    """
    读取 wav 文件，截取前 duration_sec 秒（如果指定），
    计算 FFT 并绘制幅度频谱图（纵轴为对数刻度），保存为 PDF。

    参数：
        wav_path: 输入 WAV 文件路径
        output_name: 输出 PDF 文件名前缀
        duration_sec: 可选，截取的音频时长（秒）
        freq_range_hz: 可选，显示的频率范围 (min_hz, max_hz)，如 (0, 1000)
    """
    # 1. 读取音频
    data, sr = sf.read(wav_path)
    if data.ndim > 1:
        data = np.mean(data, axis=1)

    # 2. 可选：截取前 duration_sec 秒
    if duration_sec is not None:
        max_samples = int(duration_sec * sr)
        data = data[:max_samples]
```

```

# 3. 计算 FFT
N = len(data)
Y = np.fft.rfft(data)
freqs = np.fft.rfftfreq(N, d=1/sr)
magnitude = np.abs(Y) / N

# 4. 避免对数刻度的零值问题
magnitude = np.clip(magnitude, 1e-10, None) # 将幅度限制在最小值 1e-10

# 5. 准备输出文件夹
output_dir = "Figure"
os.makedirs(output_dir, exist_ok=True)
output_path = os.path.join(output_dir, f"{output_name}.pdf")

# 6. 绘制频谱图并保存
plt.figure(figsize=(10, 6))
plt.plot(freqs, magnitude, linewidth=1)
plt.title(f"FFT Spectrum of {os.path.basename(wav_path)} (Log Scale)...")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Magnitude (Log Scale)")

# 7. 设置频率显示范围
if freq_range_hz is not None:
    min_hz, max_hz = freq_range_hz
    plt.xlim(min_hz, max_hz)
else:
    plt.xlim(0, sr / 2)

# 8. 设置纵轴为对数刻度
plt.yscale('log')
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig(output_path, format='pdf')
plt.close()

print(f"已保存频谱图到: {output_path}")

if __name__ == "__main__":
    # 用户输入文件名 (不含后缀)
    user_prefix = input("请输入输出文件名前缀 (不含后缀): ").strip()
    WAV_FILE = "notch_output/example_50Hz_notch.wav"
    # 示例: 限制频率范围为 0 到 1000 Hz
    plot_fft_spectrum(WAV_FILE, user_prefix, duration_sec=None, ...
                      freq_range_hz=(0, 100))

```

Listing 17: 生成不同采样率以及量化技术的音乐

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

```

```
"""
批量测试不同采样率和量化位深的听觉效果
依赖: numpy, librosa, soundfile
    pip install numpy librosa soundfile
"""

import os
import numpy as np
import soundfile as sf
import librosa

def resample_and_quantize(
    data: np.ndarray,
    orig_sr: int,
    target_sr: int,
    bits: int
) -> np.ndarray:
    """
    对 data 做重采样到 target_sr, 再做 bits 位量化,
    返回量化后的浮点数据 (已经在 [-1,1] 范围内)。
    """
    # 1) 重采样
    y = librosa.resample(data, orig_sr=orig_sr, target_sr=target_sr)
    # 2) 量化: 将 [-1,1] 映射到整数域, 再映射回浮点
    max_int = 2**(bits - 1) - 1
    y_q = np.round(y * max_int) / max_int
    return np.clip(y_q, -1.0, 1.0)

def batch_process(
    input_wav: str,
    sample_rates: list[int],
    bit_depths: list[int],
    output_dir: str = "output" # 修改: 默认输出目录为 output
):
    """
    读取 input_wav, 对每个采样率和位深组合:
    1. 重采样
    2. 定点量化
    3. 写入 WAV 文件 (文件名包含 fs 和 bits)
    """
    # 检查文件
    if not os.path.exists(input_wav):
        raise FileNotFoundError(f"找不到文件: {input_wav}")

    # 读取音频
    data, sr = sf.read(input_wav)
    # 如果是多通道, 转为单声道
    if data.ndim > 1:
        data = data.mean(axis=1)

    # 准备输出目录
```

```

os.makedirs(output_dir, exist_ok=True)
basename = os.path.splitext(os.path.basename(input_wav))[0]

# PCM 子类型映射
subtype_map = {
    8: "PCM_U8", # 8-bit unsigned PCM
    16: "PCM_16",
    24: "PCM_24",
    32: "PCM_32"
}

# 遍历采样率和位深组合
for fs in sample_rates:
    for bits in bit_depths:
        # 重采样 + 量化
        y_q = resample_and_quantize(data, sr, fs, bits)

        # 构造输出文件名
        out_name = f"{basename}_{fs}Hz_{bits}bit.wav"
        out_path = os.path.join(output_dir, out_name)

        # 选择正确的 subtype
        subtype = subtype_map.get(bits)
        if subtype is None:
            raise ValueError(f"不支持 {bits} 位深的 PCM 子类型")

        # 写入 WAV
        sf.write("output/"+out_path, y_q, fs, subtype=subtype)
        print(f"Saved: {out_path}")

if __name__ == "__main__":
    # 用户可根据需要修改的参数
    INPUT_WAV = "example.wav"
    SAMPLE_RATES = [8000, 16000, 22050, 44100, 48000]
    BIT_DEPTHS = [8, 16, 24]

    batch_process(INPUT_WAV, SAMPLE_RATES, BIT_DEPTHS)

```

Listing 18: 添加 50Hz 噪声并去噪

```

import numpy as np
import soundfile as sf
from scipy.signal import iirnotch, filtfilt

# 1. 读取原始音频
data, samplerate = sf.read('example.wav')

# 2. 生成 50 Hz 干扰噪声
t = np.arange(len(data)) / samplerate
if data.ndim > 1:

```



```

# 多声道时，为每个通道生成相同的噪声
noise = np.sin(2 * np.pi * 50 * t)[: , np.newaxis]
else:
    noise = np.sin(2 * np.pi * 50 * t)

# 将噪声振幅设为原始信号峰值的 10%
noise_amp = 0.1 * np.max(np.abs(data))
noisy = data + noise_amp * noise

# 保存带噪音频
sf.write('example_50Hz_noisy.wav', noisy, samplerate)

# 3. 设计 50 Hz 陷波滤波器 (Notch Filter)
f0 = 50.0 # 欲去除的频率 (Hz)
Q = 30.0 # 品质因数，数值越大陷波越窄
b, a = iirnotch(f0, Q, samplerate)

# 4. 零相位滤波以避免相位失真
filtered = filtfilt(b, a, noisy, axis=0)

# 保存去噪后音频
sf.write('example_50Hz_notch.wav', filtered, samplerate)

print("□ 已生成 example_50Hz_noisy.wav 和 example_50Hz_notch.wav")

```

Listing 19: 使用前馈梳状滤波器法生成回声

```

import numpy as np
import soundfile as sf

def add_echo(input_file, output_file, delay_ms=500, attenuation=0.6):
    # 读取音频文件
    data, samplerate = sf.read(input_file)
    print(f"采样率: {samplerate} Hz, 数据长度: {len(data)} 样点")

    # 将延迟毫秒转为样点数
    delay_samples = int(samplerate * delay_ms / 1000)
    print(f"回声延迟: {delay_ms} ms ({delay_samples} 样点)")

    # 创建输出数组 (多留出延迟部分空间)
    if data.ndim == 1: # 单声道
        echo_data = np.zeros(len(data) + delay_samples)
        echo_data[:len(data)] = data
        echo_data[delay_samples:] += attenuation * data
    else: # 双声道
        echo_data = np.zeros((len(data) + delay_samples, data.shape[1]))
        echo_data[:len(data), :] = data
        echo_data[delay_samples:, :] += attenuation * data

    # 标准化，防止溢出

```

```

max_val = np.max(np.abs(echo_data))
if max_val > 1.0:
    echo_data = echo_data / max_val

# 写入输出文件
sf.write("output/"+output_file, echo_data, samplerate)
print(f"已保存回声文件到: {output_file}")

# 示例调用
add_echo('trump_speech.wav', 'trump_speech_echo.wav', delay_ms=400, ...
         attenuation=0.45)

```

```

import numpy as np
import soundfile as sf
from scipy.signal import lfilter

def allpass_echo_fast(input_file,
                      output_file,
                      delay_ms=400,
                      alpha=0.45,
                      output_duration=None):
    """
    全通滤波器回声效果 (C 层 lfilter 实现, 速度极快)。

    参数:
        input_file  输入音频文件路径
        output_file 输出音频文件路径
        delay_ms     延迟时间 (毫秒)
        alpha        衰减系数 (0~1)
        output_duration 输出时长 (秒), 为 None 则处理整段
    """
    # 读取音频
    data, sr = sf.read(input_file)
    # 保持二维通道格式
    if data.ndim == 1:
        data = data[:, np.newaxis]

    # 如果要截断时长, 先切片
    if output_duration is not None:
        max_samps = int(sr * output_duration)
        data = data[:max_samps]

    # 延迟样本数
    D = int(sr * delay_ms / 1000)

    # 构造滤波器系数
    # 全通  $H(z) = (\alpha + z^{-D}) / (1 + \alpha z^{-D})$ 
    # 分子 b =  $[\alpha, 0, 0, \dots, 0, 1]$ 
    # 分母 a =  $[1, 0, 0, \dots, 0, \alpha]$ 
    b = np.zeros(D+1, dtype=np.float64)

```

```

b[0] = alpha
b[-1] = 1.0
a = np.zeros(D+1, dtype=np.float64)
a[0] = 1.0
a[-1] = alpha

# 对每个通道都用 lfilter 做 IIR 运算
# axis=0 保证对时间轴进行滤波
output = lfilter(b, a, data, axis=0)

# 归一化防止超标
peak = np.max(np.abs(output))
if peak > 1.0:
    output /= peak

# 写文件 (如果是单通道, 恢复一维)
if output.shape[1] == 1:
    output = output[:, 0]
sf.write(output_file, output, sr)
print(f"□ 已生成快速全通回声文件: {output_file}")

if __name__ == "__main__":
    # 示例
    allpass_echo_fast(
        input_file="trump_speech.wav",
        output_file="output/trump_speech_allpass_fast.wav",
        delay_ms=400,
        alpha=0.45,
        output_duration=600
    )

```

Listing 20: 使用快速反馈型梳状滤波器法生成回声

```

import numpy as np
import soundfile as sf
from scipy.signal import lfilter

def feedback_comb_echo_fast(input_file,
                            output_file,
                            delay_ms=400,
                            feedback=0.45,
                            output_duration=None):
    """
    快速反馈型梳状滤波器回声 (基于 scipy.signal.lfilter, C 层运算)。

    参数:
        input_file  输入音频文件路径
        output_file 输出音频文件路径
        delay_ms     延迟时间 (毫秒)
        feedback     反馈系数 (0~1)
    """

```

```
    output_duration 输出时长（秒），为 None 则处理整段
    """
    # 读取音频并保证二维通道格式
    data, sr = sf.read(input_file)
    if data.ndim == 1:
        data = data[:, np.newaxis]

    # 截断到指定时长，减少数据量
    if output_duration is not None:
        max_samps = int(sr * output_duration)
        data = data[:max_samps]

    # 延迟样本数
    D = int(sr * delay_ms / 1000)

    # 构造 IIR 梳状滤波器系数：
    #  $H(z) = 1 / (1 - \text{feedback} * z^{-D})$ 
    # 分子  $b = [1, 0, 0, \dots, 0]$ （长度  $D+1$ ）
    # 分母  $a = [1, 0, 0, \dots, -\text{feedback}]$ 
    b = np.zeros(D + 1, dtype=np.float64)
    b[0] = 1.0

    a = np.zeros(D + 1, dtype=np.float64)
    a[0] = 1.0
    a[-1] = -feedback

    # 在 C 层面一次性完成递归运算
    output = lfilter(b, a, data, axis=0)

    # 归一化防止裁剪
    peak = np.max(np.abs(output))
    if peak > 1.0:
        output /= peak

    # 单通道数据恢复一维
    if output.shape[1] == 1:
        output = output[:, 0]

    sf.write(output_file, output, sr)
    print(f"□ 已生成快速反馈型梳状滤波回声文件：{output_file}")

if __name__ == "__main__":
    feedback_comb_echo_fast(
        input_file="trump_speech.wav",
        output_file="output/trump_speech_feedback_fast.wav",
        delay_ms=400,
        feedback=0.45,
        output_duration=600
    )
```

Listing 21: 可视化智能均衡器

```
#!/usr/bin/env python3
# equalizer.py — 球=振幅 · 常数, 进度拖动松手后才 Seek

import numpy as np, sounddevice as sd, soundfile as sf, time, colorsys
import ttkbootstrap as ttk, tkinter as tk
from scipy.signal import tf2sos, sosfilt
from ttkbootstrap.constants import *

# —— IIR 设计 —— 与前相同 —— #
def shelving(fs, f0, g, kind):
    G=10**(g/20); A=np.sqrt(G); Q=1/np.sqrt(2)
    w=2*np.pi*f0/fs; a=np.sin(w)/(2*Q)
    if kind=='low':
        b0=A*((A+1)-(A-1)*np.cos(w)+2*np.sqrt(A)*a)
        b1=2*A*((A-1)-(A+1)*np.cos(w)); b2=A*((A+1)-(A-1)*np.cos(w)-2*...
            np.sqrt(A)*a)
        a0=(A+1)+(A-1)*np.cos(w)+2*np.sqrt(A)*a; a1=-2*((A-1)+(A+1)*np....
            cos(w))
        a2=(A+1)+(A-1)*np.cos(w)-2*np.sqrt(A)*a
    else:
        b0=A*((A+1)+(A-1)*np.cos(w)+2*np.sqrt(A)*a)
        b1=-2*A*((A-1)+(A+1)*np.cos(w)); b2=A*((A+1)+(A-1)*np.cos(w)-2*...
            np.sqrt(A)*a)
        a0=(A+1)-(A-1)*np.cos(w)+2*np.sqrt(A)*a; a1=2*((A-1)-(A+1)*np....
            cos(w))
        a2=(A+1)-(A-1)*np.cos(w)-2*np.sqrt(A)*a
    b=np.array([b0,b1,b2])/a0; a=np.array([1,a1/a0,a2/a0]); return ...
    tf2sos(b,a)

def peaking(fs, f0, g, Q=1):
    A=10**(g/40); w=2*np.pi*f0/fs; a=np.sin(w)/(2*Q)
    b0=1+a*A; b1=-2*np.cos(w); b2=1-a*A
    a0=1+a/A; a1=b1; a2=1-a/A
    b=np.array([b0,b1,b2])/a0; a=np.array([1,a1/a0,a2/a0]); return ...
    tf2sos(b,a)

# —— GUI —— #
class EQApp(ttk.Window):
    FS=44100; SPLIT=(150,650,5000)
    WAVE_W,WAVE_H=480,80
    def __init__(self,wav="example.wav"):
        super().__init__(themename="cosmo")
        self.title("均衡器 "); self.geometry("560x720"); self.resizable(...
            False,False)
        # === 在 __init__ 里给球动画再加两个成员 ===
        self.phi = 0.0 # 当前相位
        self.omega0 = 2 # 最小基频 (rad/s)
        self.kfreq = 8 # 响度映射系数
        self.audio,fs=sf.read(wav,always_2d=True)
        if fs!=self.FS: raise RuntimeError("需要 44.1 kHz WAV")
```

```

self.nch=self.audio.shape[1]; self.total=len(self.audio)/self.FS
self.eq_audio=self.audio
self.start_time=None; self.start_pos=0; self.playing=False

# ==== 球 ====
self.csize=220
self.ball=tk.Canvas(self,width=self.csize,height=self.csize,bg="...
    white",highlightthickness=0)
self.ball.pack(pady=10)
self.after_id=self.after(16,self._animate) # -60 fps

# ==== 波形 & 游标 ====
self.wave=tk.Canvas(self,width=self.WAVE_W,height=self.WAVE_H,bg=...
    "#111",highlightthickness=0)
self.wave.pack()
self._draw_wave()
self.cursor=self.wave.create_line(0,0,0,self.WAVE_H,fill="cyan",...
    width=2)
self.dragging=False
self.wave.bind("<Button-1>",self._drag_start)
self.wave.bind("<B1-Motion>",self._drag_move)
self.wave.bind("<ButtonRelease-1>",self._drag_end)

# ==== 滑块 ====
self._slider("低频增益 (dB)","low",0)
self._slider("中频增益 (dB)","mid",0)
self._slider("高频增益 (dB)","high",0)

# ==== Buttons ====
f=ttk.Frame(self); f.pack(pady=20)
ttk.Button(f,text="应用均衡并播放",bootstyle="success-outline",...
    command=self.apply).grid(row=0,column=0,padx=12)
ttk.Button(f,text="暂停 / 继续",bootstyle="warning-outline",...
    command=self.toggle).grid(row=0,column=1,padx=12)

self.prog=self.after(100,self._update_cursor)
self.protocol("WM_DELETE_WINDOW",self._close)

# --- basic UI helpers ---
def _slider(self,text,key,val):
    var=ttk.DoubleVar(value=val); setattr(self,f"g_{key}",var)
    ttk.Label(self,text=text,font=("Helvetica",11,"bold")).pack(pady...
        =(5,0))
    ttk.Scale(self,from_=-12,to=12,variable=var,length=480,bootstyle=...
        "info",
        command=lambda *_: getattr(self,f"lab_{key}").config(text...
            =f"{var.get():.1f} dB")).pack()
    lab=ttk.Label(self,text=f"{val:.1f} dB"); lab.pack(); setattr(...
        self,f"lab_{key}",lab)

# --- waveform draw ---

```

```

def _draw_wave(self):
    hop=len(self.audio)//self.WAVE_W
    env=np.sqrt(np.mean(self.audio[:self.WAVE_W*hop:hop]**2,axis=1))...
        ; env/=env.max(); env=1-env
    pts=[(x,env[x]*(self.WAVE_H-2)+1) for x in range(self.WAVE_W)]
    flat=[p for xy in pts for p in xy]; self.wave.create_line(flat,...
        fill="#55ff55")
def _place_cursor(self,frac):
    x=max(0,min(frac*self.WAVE_W,self.WAVE_W)); self.wave.coords(self...
        .cursor,x,0,x,self.WAVE_H)

# --- dragging ---
def _drag_start(self,e):
    self.dragging=True; self._drag_move(e)
def _drag_move(self,e):
    if not self.dragging: return
    frac=min(max(e.x,0),self.WAVE_W)/self.WAVE_W
    self._place_cursor(frac)
def _drag_end(self,e):
    frac=min(max(e.x,0),self.WAVE_W)/self.WAVE_W
    self.start_pos=frac*self.total
    self._place_cursor(frac)
    self.dragging=False
    if self.playing:
        self.start_time=time.monotonic()
        sd.stop(); sd.play(self.eq_audio[int(self.start_pos*self.FS)...
            :],self.FS)

# --- 球动画 (振幅驱动) ---
def _animate(self):
    # 1. 取最近 50 ms 片段的 RMS (响度)
    idx = int(self._now() * self.FS)
    seg = self.eq_audio[idx : idx + int(0.05 * self.FS)]
    rms = float(np.sqrt(np.mean(seg ** 2))) if seg.size else 0.0

    # 2. 根据响度决定相位增量 (速度) 并更新相位
    omega = self.omega0 / 2 + self.kfreq * rms * 2 # rad per second
    self.phi += omega * 0.016 # Δt ≈ 16 ms
    self.phi %= 2 * np.pi

    # 3. 球半径: base + amp * sin(phi) (amp 固定不再随 rms)
    base, amp = 70, 35
    r = base + amp * np.sin(self.phi)

    # 4. 颜色依旧按时间彩虹
    hue = (time.perf_counter() * 0.2) % 1
    rgb = colorsys.hsv_to_rgb(hue, 1, 1)
    col = f"#{int(rgb[0]*255):02x}{int(rgb[1]*255):02x}{int(rgb...
        [2]*255):02x}"

    # 5. 重新绘制

```



```

        c = self.csize / 2
        self.ball.delete("all")
        self.ball.create_oval(c - r, c - r, c + r, c + r, fill=col, ...
                               outline="")

# 6. 下一帧
self.after_id = self.after(16, self._animate)

# --- playback helpers ---
def _now(self):
    return self.start_pos+(time.monotonic()-self.start_time) if self...
        playing else self.start_pos
def apply(self):
    if self.playing: self.start_pos=self._now()
    gl,gm,gh=(self.g_low.get(),self.g_mid.get(),self.g_high.get())
    sos=np.vstack([shelving(self.FS,200,gl,"low"), peaking(self.FS...
        ,1000,gm), shelving(self.FS,5000,gh,"high")])
    y=sosfilt(sos,self.audio,axis=0); y/=max(1,np.max(np.abs(y)))
    self.eq_audio=np.ascontiguousarray(y,dtype=np.float32)
    sd.stop(); sd.play(self.eq_audio[int(self.start_pos*self.FS):],...
        self.FS)
    self.start_time=time.monotonic(); self.playing=True
def toggle(self):
    if self.playing:
        self.start_pos=self._now(); sd.stop(); self.playing=False
    else:
        sd.play(self.eq_audio[int(self.start_pos*self.FS):],self.FS)
        self.start_time=time.monotonic(); self.playing=True

# --- cursor updater ---
def _update_cursor(self):
    if not self.dragging: self._place_cursor(self._now()/self.total...
        %1)
    self.prog=self.after(100,self._update_cursor)

# --- close ---
def _close(self):
    sd.stop()
    for attr in ("after_id","prog"):
        if hasattr(self,attr): self.after_cancel(getattr(self,attr))
    self.destroy()

if __name__=="__main__":
    EQApp("example.wav").mainloop()

```

Listing 22: DSP 方法进行音频风格迁移

```

#!/usr/bin/env python3
"""
style_audio_player.py — 播放 / 暂停 + 七种音色一键切换

```

(电话 / 老唱片 / 机器人 / 电台 / 演唱会 / 低音增强 / 原始)

去除了原均衡器功能, 自动截取音频文件的前 5 分钟。

依赖: numpy, soundfile, sounddevice, scipy, ttkbootstrap

"""

```
import numpy as np
import sounddevice as sd
import soundfile as sf
import time, colorsys
import ttkbootstrap as ttk
import tkinter as tk
from scipy.signal import butter, lfilter, sosfiltfilt
from ttkbootstrap.constants import *
```

```
# ----- 音色效果函数 -----

def _normalize(y: np.ndarray) -> np.ndarray:
    """防止溢出, 统一幅度"""
    peak = np.max(np.abs(y))
    return y / peak if peak > 0 else y

def phone_effect(x, fs):
    """带通 300-3400 Hz, 模拟电话音色"""
    nyq = fs * 0.5
    b, a = butter(6, [300 / nyq, 3400 / nyq], btype="band")
    y = lfilter(b, a, x, axis=0)
    return _normalize(y)

def vintage_effect(x, *_):
    """加入底噪 + 轻微剪切, 模拟老唱片"""
    noise = np.random.normal(0, 0.01, x.shape)
    y = np.clip(x + noise, -0.7, 0.7)
    return _normalize(y)

def robotic_effect(x, *_):
    """16 级量化, 模拟机器人音"""
    levels = 16
    y = np.round(x * levels) / levels
    return _normalize(y)

def radio_effect(x, fs):
    """带通过度更高, 附加轻微压缩, 模拟调频电台"""
    nyq = fs * 0.5
    b, a = butter(4, [500 / nyq, 5000 / nyq], btype="band")
    y = lfilter(b, a, x, axis=0)
    # 简易压缩
    y = np.tanh(2 * y)
```

```
return _normalize(y)

def hall_effect(x, fs):
    """简易混响：多次延迟叠加，模拟演唱会大厅"""
    delays_ms = [20, 40, 60, 80]
    decays = [0.6, 0.5, 0.4, 0.3]
    y = x.astype(np.float32).copy()
    for d_ms, g in zip(delays_ms, decays):
        d = int(fs * d_ms / 1000)
        echo = np.zeros_like(y)
        echo[d:] = x[:-d] * g
        y += echo
    return _normalize(y)

def bassboost_effect(x, fs):
    """低频提升，适合低音党"""
    nyq = fs * 0.5
    sos = butter(4, 180 / nyq, btype="low", output="sos")
    low = sosfiltfilt(sos, x, axis=0)
    y = x + 1.5 * low # 低频增益 1.5 倍
    return _normalize(y)

# ----- GUI 主类 -----

class StylePlayer(ttk.Window):
    FS = 44100
    WAVE_W, WAVE_H = 480, 80

    FX_LIST = [
        ("电话", phone_effect),
        ("老唱片", vintage_effect),
        ("机器人", robotic_effect),
        ("电台", radio_effect),
        ("演唱会", hall_effect),
        ("低音增强", bassboost_effect),
        ("原始", lambda x, *_: x),
    ]

    def __init__(self, wav="trump_speech.wav"):
        super().__init__(themename="cosmo")
        self.title("Style Audio Player")
        self.geometry("560x760")
        self.resizable(False, False)

        # === 音频加载（截取前 5 min） ===
        audio, fs = sf.read(wav, always_2d=True)
        if fs != self.FS:
            raise RuntimeError("需要 44.1 kHz WAV")
        max_samples = 5 * 60 * self.FS
```

```

self.base_audio = audio[:max_samples]
self.audio = self.base_audio.copy()
self.total = len(self.audio) / self.FS
self.nch = self.audio.shape[1]
self.start_time = None
self.start_pos = 0
self.playing = False
self.current_fx = "原始"

# === 呼吸球动画 ===
self.csize = 220
self.phi = 0.0; self.omega0 = 2; self.kfreq = 8
self.ball = tk.Canvas(self, width=self.csize, height=self.csize, ...
    bg="white", highlightthickness=0)
self.ball.pack(pady=10)
self.after_id = self.after(16, self._animate)

# === 波形与游标 ===
self.wave = tk.Canvas(self, width=self.WAVE_W, height=self.WAVE_H...
    , bg="#111", highlightthickness=0)
self.wave.pack()
self._draw_wave()
self.cursor = self.wave.create_line(0, 0, 0, self.WAVE_H, fill="...
    cyan", width=2)
self.dragging = False
self.wave.bind("<Button-1>", self._drag_start)
self.wave.bind("<B1-Motion>", self._drag_move)
self.wave.bind("<ButtonRelease-1>", self._drag_end)

# === 音色按钮 ===
fx_frame = ttk.Frame(self); fx_frame.pack(pady=12)
ttk.Label(fx_frame, text="音色风格化", font=("Helvetica", 12, "bold...
    ")).grid(row=0, column=0, columnspan=4, pady=(0, 6))
# 第一行 4 个
for i, (name, func) in enumerate(self.FX_LIST[:4]):
    ttk.Button(fx_frame, text=name, width=10, bootstyle="secondary...
        -outline", command=lambda f=func, n=name: self.set_style(f,...
            n)).grid(row=1, column=i, padx=6)
# 第二行 3 个
for i, (name, func) in enumerate(self.FX_LIST[4:7]):
    style = "danger-outline" if name == "原始" else "secondary-...
        outline"
    ttk.Button(fx_frame, text=name, width=10, bootstyle=style, ...
        command=lambda f=func, n=name: self.set_style(f, n)).grid(...
        row=2, column=i+0, padx=6, pady=4)

# === 播放控制 ===
ctrl = ttk.Frame(self); ctrl.pack(pady=20)
ttk.Button(ctrl, text="播放", bootstyle="success-outline", command...
    =self.play_from_start).grid(row=0, column=0, padx=12)
ttk.Button(ctrl, text="暂停 / 继续", bootstyle="warning-outline", ...

```

```

        command=self.toggle).grid(row=0, column=1, padx=12)

self.prog = self.after(100, self._update_cursor)
self.protocol("WM_DELETE_WINDOW", self._close)

# ----- 波形绘制与拖动 -----
def _draw_wave(self):
    self.wave.delete("all")
    hop = len(self.audio) // self.WAVE_W
    env = np.sqrt(np.mean(self.audio[:self.WAVE_W*hop:hop] ** 2, ...
        axis=1))
    env /= env.max(); env = 1 - env
    pts = [(x, env[x]*(self.WAVE_H-2) + 1) for x in range(self.WAVE_W...
        )]
    flat = [p for xy in pts for p in xy]
    self.wave.create_line(flat, fill="#55ff55")

def _place_cursor(self, frac):
    x = max(0, min(frac * self.WAVE_W, self.WAVE_W))
    self.wave.coords(self.cursor, x, 0, x, self.WAVE_H)

def _drag_start(self, e):
    self.dragging = True; self._drag_move(e)
def _drag_move(self, e):
    if not self.dragging: return
    frac = min(max(e.x, 0), self.WAVE_W) / self.WAVE_W
    self._place_cursor(frac)
def _drag_end(self, e):
    frac = min(max(e.x, 0), self.WAVE_W) / self.WAVE_W
    self.start_pos = frac * self.total
    self._place_cursor(frac)
    self.dragging = False
    if self.playing:
        self.start_time = time.monotonic()
        sd.stop(); sd.play(self.audio[int(self.start_pos*self.FS):], ...
            self.FS)

# ----- 呼吸球动画 -----
def _animate(self):
    idx = int(self._now() * self.FS)
    seg = self.audio[idx:idx + int(0.05 * self.FS)]
    rms = float(np.sqrt(np.mean(seg ** 2))) if seg.size else 0.0
    omega = self.omega0 / 2 + self.kfreq * rms * 2
    self.phi = (self.phi + omega * 0.016) % (2*np.pi)
    base, amp = 70, 35; r = base + amp * np.sin(self.phi)

    hue = (time.perf_counter() * 0.2) % 1
    rgb = colorsys.hsv_to_rgb(hue, 1, 1)
    col = f"#{int(rgb[0]*255):02x}{int(rgb[1]*255):02x}{int(rgb...
        [2]*255):02x}"

```

```
c = self.csize / 2
self.ball.delete("all")
self.ball.create_oval(c - r, c - r, c + r, c + r, fill=col, ...
    outline="")
self.after_id = self.after(16, self._animate)

# ----- 音频控制 -----
def set_style(self, fx_func, name):
    self.current_fx = name
    self.audio = np.ascontiguousarray(fx_func(self.base_audio, self...
        FS), dtype=np.float32)
    self.total = len(self.audio) / self.FS
    self._draw_wave(); self._restart_playback()

def play_from_start(self):
    self.start_pos = 0; self._restart_playback()

def _restart_playback(self):
    sd.stop(); sd.play(self.audio[int(self.start_pos*self.FS):], self...
        .FS)
    self.start_time = time.monotonic(); self.playing = True

def _now(self):
    return self.start_pos + (time.monotonic() - self.start_time) if ...
        self.playing else self.start_pos

def toggle(self):
    if self.playing:
        self.start_pos = self._now(); sd.stop(); self.playing = False
    else:
        sd.play(self.audio[int(self.start_pos*self.FS):], self.FS)
        self.start_time = time.monotonic(); self.playing = True

def _update_cursor(self):
    if not self.dragging:
        self._place_cursor(self._now() / self.total % 1)
    self.prog = self.after(100, self._update_cursor)

# ----- 关闭 -----
def _close(self):
    sd.stop()
    for attr in ("after_id", "prog"):
        if hasattr(self, attr):
            self.after_cancel(getattr(self, attr))
    self.destroy()

# ----- 运行 -----
if __name__ == "__main__":
    stylePlayer("trump_speech.wav").mainloop()
```

Listing 23: DSP 方法进行音频风格迁移结果的 STFT 图像

```
#!/usr/bin/env python3
"""
style_audio_stft.py — generate STFT images for seven timbre effects
Dependencies: numpy, soundfile, scipy, matplotlib
"""

import os
import numpy as np
import soundfile as sf
import matplotlib.pyplot as plt
from scipy.signal import butter, lfilter, sosfiltfilt, stft

# ——— helper ———
def _normalize(y: np.ndarray) -> np.ndarray:
    peak = np.max(np.abs(y))
    return y / peak if peak > 0 else y

# ——— effects ———
def phone_effect(x, fs):
    nyq = 0.5 * fs
    b, a = butter(6, [300 / nyq, 3400 / nyq], btype="band")
    return _normalize(lfilter(b, a, x, axis=0))

def vintage_effect(x, *_):
    noise = np.random.normal(0, 0.01, x.shape)
    return _normalize(np.clip(x + noise, -0.7, 0.7))

def robotic_effect(x, *_):
    levels = 16
    return _normalize(np.round(x * levels) / levels)

def radio_effect(x, fs):
    nyq = 0.5 * fs
    b, a = butter(4, [500 / nyq, 5000 / nyq], btype="band")
    return _normalize(np.tanh(2 * lfilter(b, a, x, axis=0)))

def hall_effect(x, fs):
    delays_ms = [20, 40, 60, 80]
    decays = [0.6, 0.5, 0.4, 0.3]
    y = x.astype(np.float32).copy()
    for d_ms, g in zip(delays_ms, decays):
        d = int(fs * d_ms / 1000)
        y[d:] += x[:-d] * g
    return _normalize(y)

def bassboost_effect(x, fs):
    nyq = 0.5 * fs
    sos = butter(4, 180 / nyq, btype="low", output="sos")
    return _normalize(x + 1.5 * sosfiltfilt(sos, x, axis=0))
```

```
FX_LIST = [  
    ("phone", phone_effect),  
    ("vintage", vintage_effect),  
    ("robotic", robotic_effect),  
    ("radio", radio_effect),  
    ("hall", hall_effect),  
    ("bassboost", bassboost_effect),  
    ("original", lambda x, *_: x),  
]  
  
# ----- main -----  
def main(wav="trump_speech.wav", out_dir="Figure", seg_sec=10):  
    os.makedirs(out_dir, exist_ok=True)  
    audio, fs = sf.read(wav, always_2d=True)  
    if fs != 44_100:  
        raise RuntimeError("Requires 44.1 kHz WAV")  
    seg_len = int(seg_sec * fs)  
  
    for tag, fx in FX_LIST:  
        proc = fx(audio, fs)[:seg_len].mean(axis=1) # mono  
        f, t, Z = stft(proc, fs=fs, nperseg=1024, noverlap=768, window="...  
            hann")  
        mag_db = 20 * np.log10(np.abs(Z) + 1e-9)  
  
        plt.figure(figsize=(6, 4))  
        plt.pcolormesh(t, f, mag_db, shading="gouraud")  
        plt.title(f"{tag.capitalize()} — STFT")  
        plt.ylabel("Frequency [Hz]")  
        plt.xlabel("Time [s]")  
        plt.colorbar(label="Amplitude [dB]")  
        plt.tight_layout()  
        out_path = os.path.join(out_dir, f"{tag}_stft.png")  
        plt.savefig(out_path, dpi=200)  
        plt.close()  
        print(f"Saved: {out_path}")  
  
    print("Done.")  
  
if __name__ == "__main__":  
    main()
```